Linguagem de Programação II - Etapa 4 - Leonardo Farias de Oliveira
$$$ sql.banco

```
DROP TABLE IF EXISTS peças_sinistros;
DROP TABLE IF EXISTS `Orçamentos`;
DROP TABLE IF EXISTS peça_mecânica;
DROP TABLE IF EXISTS peça_acessórios;
DROP TABLE IF EXISTS peças;
DROP TABLE IF EXISTS sinistros;
DROP TABLE IF EXISTS seguradoras;

CREATE TABLE seguradoras (
    nome VARCHAR(50) PRIMARY KEY,
    cidade VARCHAR(50),
    cobertura_percentual INT,
    possui_atendimento_24h BOOLEAN,
    forma_pagamento_preferencial VARCHAR(20)
);

CREATE TABLE sinistros (
    id INT NOT NULL AUTO_INCREMENT PRIMARY KEY,
    segurado VARCHAR(80) NOT NULL,
    telefone VARCHAR(20),
    cidade VARCHAR(50),
    grau_monta VARCHAR(20),
    perda_total BOOLEAN
);

CREATE TABLE peças (
    codigo INT PRIMARY KEY,
    nome VARCHAR(50),
    marca VARCHAR(20),
    preço DECIMAL(10,2),
    sob_encomenda BOOLEAN NOT NULL DEFAULT FALSE
);

CREATE TABLE peças_sinistros (
```

```
    peça_codigo INT NOT NULL,
    sinistro_id INT NOT NULL,
    PRIMARY KEY (peça_codigo, sinistro_id),
    FOREIGN KEY (peça_codigo) REFERENCES peças(codigo) ON DELETE CASCADE,
    FOREIGN KEY (sinistro_id) REFERENCES sinistros(id) ON DELETE CASCADE
);

CREATE TABLE peça_mecânica (
    categoria_mecânica VARCHAR(40) NOT NULL,
    prazo_garantia INT,
    mecanico_proprio BOOLEAN NOT NULL DEFAULT FALSE,
    codigoId INT NOT NULL,
    FOREIGN KEY (codigoId) REFERENCES peças (codigo)
);

CREATE TABLE peça_acessórios (
    categoria_acessorio VARCHAR(40) NOT NULL,
    cor VARCHAR(20),
    funileiro_proprio BOOLEAN NOT NULL DEFAULT FALSE,
    codigoId INT NOT NULL,
    FOREIGN KEY (codigoId) REFERENCES peças (codigo)
);

CREATE TABLE Orçamentos (
    data TIMESTAMP NOT NULL,
    sinistro INT NOT NULL,
    seguradora VARCHAR(50) NOT NULL,
    -- Enumerado: aprovado, pendente ou recusado.
    status VARCHAR(10) NOT NULL,
    valor_total DECIMAL(10,2) NOT NULL,
    possui_danos_ocultos BOOLEAN NOT NULL DEFAULT FALSE,
    PRIMARY KEY (sinistro, seguradora),
    FOREIGN KEY (sinistro) REFERENCES sinistros(id),
    FOREIGN KEY (seguradora) REFERENCES seguradoras(nome)
);
```

$$$ src.controles.ControladorCadastroOrçamentos

```
package controles;

import entidades.Orçamento;
import interfaces.JanelaCadastroOrçamentos;

public class ControladorCadastroOrçamentos {

   public ControladorCadastroOrçamentos() {
      JanelaCadastroOrçamentos janela = new JanelaCadastroOrçamentos(this);
      janela.setVisible(true);
      janela.toFront();
      janela.requestFocus();
   }

   public String inserirOrçamento(Orçamento orçamento) {
      if (Orçamento.existeOrçamento(orçamento.getSinistro().getId(),
orçamento.getSeguradora().getNome())) {
         return "Já existe orçamento para este sinistro e seguradora";
      }
      return Orçamento.inserirOrçamento(orçamento);
   }

   public String alterarOrçamento(Orçamento orçamento, int idSinistroOriginal, String
seguradoraOriginal) {
      if (orçamento.getSinistro().getId() != idSinistroOriginal
            || !orçamento.getSeguradora().getNome().equals(seguradoraOriginal)) {
         return "Alteração não permitida: sinistro e seguradora identificam o orçamento";
      }
      if (!Orçamento.existeOrçamento(idSinistroOriginal, seguradoraOriginal)) {
         return "Orçamento não cadastrado";
      }
      return Orçamento.alterarOrçamento(orçamento);
   }

   public String removerOrçamento(int idSinistro, String nomeSeguradora) {
```

```
        if (!Orçamento.existeOrçamento(idSinistro, nomeSeguradora)) {
            return "Orçamento não cadastrado";
        }
        return Orçamento.removerOrçamento(idSinistro, nomeSeguradora);
    }
}
```

$$$ src.controles.ControladorCadastroPeças

```
package controles;

import entidades.Peça;
import interfaces.JanelaCadastroPeças;
import java.awt.Frame;

public class ControladorCadastroPeças {

   public ControladorCadastroPeças() {
      this(null);
   }

   public ControladorCadastroPeças(Frame owner) {
      JanelaCadastroPeças janela = new JanelaCadastroPeças(this, owner);
      janela.setVisible(true);
      janela.toFront();
      janela.requestFocus();
   }

   public String inserirPeças(Peça pecas) {
      Peça pecaBuscada = Peça.buscarPeças(pecas.getCodigo());
      if (pecaBuscada == null) return Peça.inserirPeças(pecas);
      else return "Código de Peça já cadastrado";
   }

   public String alterarPeças(Peça pecas) {
      Peça pecaBuscada = Peça.buscarPeças(pecas.getCodigo());
      if (pecaBuscada != null) return Peça.alterarPeças(pecas);
      else return "Código de Peça não cadastrado";
   }

   public String removerPeças(int codigo) {
      Peça pecaBuscada = Peça.buscarPeças(codigo);
      if (pecaBuscada != null) return Peça.removerPeças(codigo);
      else return "Código de Peça não cadastrado";
```

```
    }
}
```

$$$ src.controles.ControladorCadastroPeçasSinistros

```
package controles;

import entidades.Peça;
import entidades.PeçaSinistro;
import entidades.Sinistro;
import interfaces.JanelaCadastroPeçasSinistros;
import java.awt.Frame;

public class ControladorCadastroPeçasSinistros {

   public ControladorCadastroPeçasSinistros() {
      this(null, null);
   }

   public ControladorCadastroPeçasSinistros(Frame owner, Sinistro sinistro) {
      JanelaCadastroPeçasSinistros janela =
            new JanelaCadastroPeçasSinistros(this, sinistro, owner);
      janela.setVisible(true);
      janela.toFront();
      janela.requestFocus();
   }

   public String inserirPeçasSinistros(Peça peca, Sinistro sinistro) {
      Sinistro sinistroBuscado =
            sinistro != null ? Sinistro.buscarSinistro(sinistro.getId()) : null;
      if (sinistroBuscado == null) return "Sinistro nao cadastrado";

      Peça pecaBuscada =
            peca != null ? Peça.buscarPeças(peca.getCodigo()) : null;
      if (pecaBuscada == null) return "Peça nao cadastrada";

      if (PeçaSinistro.existePeçasSinistros(peca.getCodigo(), sinistro.getId())) {
         return "Peça ja associada ao sinistro";
      }
```

```
        return PeçaSinistro.inserirPeçasSinistros(pecaBuscada, sinistroBuscado);
    }

    public String removerPeçasSinistros(Peça peca, Sinistro sinistro) {
        Sinistro sinistroBuscado =
                sinistro != null ? Sinistro.buscarSinistro(sinistro.getId()) : null;
        if (sinistroBuscado == null) return "Sinistro nao cadastrado";

        Peça pecaBuscada =
                peca != null ? Peça.buscarPeças(peca.getCodigo()) : null;
        if (pecaBuscada == null) return "Peça nao cadastrada";

        if (!PeçaSinistro.existePeçasSinistros(peca.getCodigo(), sinistro.getId())) {
            return "Peça nao associada ao sinistro";
        }

        return PeçaSinistro.removerPeçasSinistros(pecaBuscada, sinistroBuscado);
    }
}
```

$$$ src.controles.ControladorCadastroSeguradoras

```
package controles;

import entidades.Seguradora;
import interfaces.JanelaCadastroSeguradoras;
import java.awt.Frame;

public class ControladorCadastroSeguradoras {

   public ControladorCadastroSeguradoras() {
      this(null);
   }

   public ControladorCadastroSeguradoras(Frame owner) {
      JanelaCadastroSeguradoras janela = new JanelaCadastroSeguradoras(this, owner);
      janela.setVisible(true);
      janela.toFront();
      janela.requestFocus();
   }

   public String inserirSeguradora(Seguradora seguradora) {
      Seguradora seguradoraBuscada = Seguradora.buscarSeguradora(seguradora.getNome());
      if (seguradoraBuscada == null) return Seguradora.inserirSeguradora(seguradora);
      else return "Nome de Seguradora já cadastrado";
   }

   public String alterarSeguradora(Seguradora seguradora) {
      Seguradora seguradoraBuscada = Seguradora.buscarSeguradora(seguradora.getNome());
      if (seguradoraBuscada != null) return Seguradora.alterarSeguradora(seguradora);
      else return "Nome de Seguradora não cadastrado";
   }

   public String removerSeguradora(String nome) {
      Seguradora seguradoraBuscada = Seguradora.buscarSeguradora(nome);
      if (seguradoraBuscada != null) return Seguradora.removerSeguradora(nome);
      else return "Nome de Seguradora não cadastrado";
```

```
    }
}
```

$$$ src.controles.ControladorCadastroSinistros

```
package controles;

import entidades.Sinistro;
import interfaces.JanelaCadastroSinistros;
import java.awt.Frame;

public class ControladorCadastroSinistros {

   public ControladorCadastroSinistros() {
      this(null);
   }

   public ControladorCadastroSinistros(Frame owner) {
      JanelaCadastroSinistros janela = new JanelaCadastroSinistros(this, owner);
      janela.setVisible(true);
      janela.toFront();
      janela.requestFocus();
   }

   public String inserirSinistro(Sinistro sinistro) {
      return Sinistro.inserirSinistro(sinistro);
   }

   public String alterarSinistro(Sinistro sinistro) {
      Sinistro sinistro1 = Sinistro.buscarSinistro(sinistro.getId());
      if (sinistro1 != null) return Sinistro.alterarSinistro(sinistro);
      else return "Segurado de Sinistro não cadastrado";
   }

   public String removerSinistro(int id) {
      Sinistro sinistro1 = Sinistro.buscarSinistro(id);
      if (sinistro1 != null) return Sinistro.removerSinistro(id);
      else return "Sinistro não cadastrado";
   }
}
```

$$$ src.entidades.Orçamento

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Timestamp;
import java.util.ArrayList;
import persistência.BD;

/** Orçamento identificado pela associação entre um sinistro e uma seguradora. */
public class Orçamento {

   public enum Status {
      aprovado("aprovado"), pendente("pendente"), recusado("recusado");
      private final String texto;
      Status(String texto) { this.texto = texto; }
      public static Status fromTexto(String texto) {
         if (texto != null) for (Status status : values())
            if (status.texto.equalsIgnoreCase(texto.trim())) return status;
         throw new IllegalArgumentException("Status de orçamento inválido: " + texto);
      }
      @Override public String toString() { return texto; }
   }

   private Status status;
   private Timestamp data;
   private Sinistro sinistro;
   private Seguradora seguradora;
   private double valorTotal;
   private boolean possuiDanosOcultos;

   public Orçamento(Status status, Timestamp data, Sinistro sinistro, Seguradora seguradora,
         double valorTotal, boolean possuiDanosOcultos) {
      this.status = status;
      this.data = data;
```

```
        this.sinistro = sinistro;
        this.seguradora = seguradora;
        this.valorTotal = valorTotal;
        this.possuiDanosOcultos = possuiDanosOcultos;
    }

    public Status getStatus() { return status; }
    public Timestamp getData() { return data; }
    public Sinistro getSinistro() { return sinistro; }
    public Seguradora getSeguradora() { return seguradora; }
    public double getValorTotal() { return valorTotal; }
    public boolean getPossuiDanosOcultos() { return possuiDanosOcultos; }
    public boolean isPossuiDanosOcultos() { return possuiDanosOcultos; }
    public void setStatus(Status status) { this.status = status; }
    public void setData(Timestamp data) { this.data = data; }
    public void setValorTotal(double valorTotal) { this.valorTotal = valorTotal; }
    public void setPossuiDanosOcultos(boolean possuiDanosOcultos) { this.possuiDanosOcultos =
possuiDanosOcultos; }

    public Orçamento getVisao() {
        return new Orçamento(status, data, sinistro, seguradora, valorTotal, possuiDanosOcultos);
    }

    @Override public String toString() {
        return data + " - Sinistro #" + sinistro.getId() + " - " + seguradora.getNome() + " (" + status +
")";
    }

    public String toStringFull() {
        StringBuilder texto = new StringBuilder();
        texto.append(status).append(" - ")
                .append(formatarData(data)).append(" - R$ ")
                .append(String.format(java.util.Locale.forLanguageTag("pt-BR"), "%.2f", valorTotal))
                .append(" - danos ocultos: ").append(possuiDanosOcultos ? "sim" : "não")
                .append(" --- ")
                .append("#").append(sinistro.getId()).append(" - ")
                .append(sinistro.getSegurado()).append(" - ")
```

```
            .append(sinistro.getCidade()).append(" - ")
            .append(sinistro.getTelefone()).append(" - ")
            .append(sinistro.getGrauMonta()).append(" - perda total: ")
            .append(sinistro.isPerdaTotal() ? "sim" : "não")
            .append("\n\t--- ")
            .append(seguradora.getNome()).append("[").append(seguradora.getCidade()).append("] -
")
            .append(seguradora.getCoberturaPercentual()).append("% - atendimento 24h: ")
            .append(seguradora.isPossuiAtendimento24h() ? "sim" : "não").append(" - ")
            .append(seguradora.getFormaPagamentoPreferencial());

        Peça[] pecas = sinistro.getPeças();
        if (pecas.length == 0) {
            texto.append("\n\t--- sem peças");
        } else {
            for (Peça peca : pecas) {
                texto.append("\n\t--- ").append(formatarPeça(peca));
            }
        }
        return texto.toString().trim();
    }

    private String formatarPeça(Peça peca) {
        StringBuilder texto = new StringBuilder();
        texto.append("#").append(peca.getCodigo()).append(" - ")
            .append(peca.getNome()).append("[").append(peca.getMarca()).append("] - R$ ")
            .append(String.format(java.util.Locale.forLanguageTag("pt-BR"), "%.2f",
peca.getPreco()))
            .append(" - sob encomenda: ").append(peca.isSobEncomenda() ? "sim" : "não");

        if (peca instanceof PeçaMecânica) {
            PeçaMecânica mecânica = (PeçaMecânica) peca;
            texto.append(" - mecânica - ").append(mecânica.getCategoriaMecânica())
                .append(" - ").append(mecânica.getPrazoGarantia()).append(" dias - mecânico próprio:
")
                .append(mecânica.isMecanicoProprio() ? "sim" : "não");
        } else if (peca instanceof PeçaAcessório) {
            PeçaAcessório acessório = (PeçaAcessório) peca;
```

```
            texto.append(" - acessório - ").append(acessório.getCategoriaAcessório())
                    .append(" - ").append(acessório.getCor()).append(" - funileiro próprio: ")
                    .append(acessório.isFunileiroProprio() ? "sim" : "não");
        }
        return texto.toString();
    }

    private String formatarData(Timestamp valor) {
        if (valor == null) return "";
        return new java.text.SimpleDateFormat("dd/MM/yyyy HH:mm'hs'").format(valor);
    }

    private static Orçamento criar(ResultSet rs) throws SQLException {
        int idSinistro = rs.getInt("sinistro");
        String nomeSeguradora = rs.getString("seguradora");
        return new Orçamento(Status.fromTexto(rs.getString("status")), rs.getTimestamp("data"),
                Sinistro.buscarSinistro(idSinistro), Seguradora.buscarSeguradora(nomeSeguradora),
                rs.getDouble("valor_total"), rs.getBoolean("possui_danos_ocultos"));
    }

    public static Orçamento[] getVisoes() {
        ArrayList<Orçamento> itens = new ArrayList<>();
        String sql = "SELECT data, sinistro, seguradora, status, valor_total, possui_danos_ocultos
FROM Orçamentos ORDER BY data, sinistro, seguradora";
        try (PreparedStatement ps = BD.conexao.prepareStatement(sql); ResultSet rs =
ps.executeQuery()) {
            while (rs.next()) itens.add(criar(rs));
        } catch (SQLException | IllegalArgumentException e) { e.printStackTrace(); }
        return itens.toArray(new Orçamento[0]);
    }

    public static Orçamento buscarOrçamento(int idSinistro, String nomeSeguradora) {
        String sql = "SELECT data, sinistro, seguradora, status, valor_total, possui_danos_ocultos
FROM Orçamentos WHERE sinistro = ? AND seguradora = ?";
        try (PreparedStatement ps = BD.conexao.prepareStatement(sql)) {
            ps.setInt(1, idSinistro);
            ps.setString(2, nomeSeguradora);
            try (ResultSet rs = ps.executeQuery()) { return rs.next() ? criar(rs) : null; }
```

```
    } catch (SQLException | IllegalArgumentException e) { e.printStackTrace(); return null; }
  }

  public static boolean existeOrçamento(int idSinistro, String nomeSeguradora) {
    try (PreparedStatement ps = BD.conexao.prepareStatement(
        "SELECT 1 FROM Orçamentos WHERE sinistro = ? AND seguradora = ?")) {
      ps.setInt(1, idSinistro);
      ps.setString(2, nomeSeguradora);
      try (ResultSet rs = ps.executeQuery()) { return rs.next(); }
    } catch (SQLException e) { e.printStackTrace(); return false; }
  }

  public static String inserirOrçamento(Orçamento o) {
    String sql = "INSERT INTO Orçamentos (data, sinistro, seguradora, status, valor_total,
possui_danos_ocultos) VALUES (?, ?, ?, ?, ?, ?)";
    try (PreparedStatement ps = BD.conexao.prepareStatement(sql)) {
      ps.setTimestamp(1, o.data); ps.setInt(2, o.sinistro.getId()); ps.setString(3,
o.seguradora.getNome());
      ps.setString(4, o.status.toString()); ps.setDouble(5, o.valorTotal); ps.setBoolean(6,
o.possuiDanosOcultos);
      ps.executeUpdate(); return null;
    } catch (SQLException e) { e.printStackTrace(); return "Erro ao inserir orçamento no banco de
dados"; }
  }

  public static String alterarOrçamento(Orçamento o) {
    String sql = "UPDATE Orçamentos SET data = ?, status = ?, valor_total = ?,
possui_danos_ocultos = ? WHERE sinistro = ? AND seguradora = ?";
    try (PreparedStatement ps = BD.conexao.prepareStatement(sql)) {
      ps.setTimestamp(1, o.data); ps.setString(2, o.status.toString()); ps.setDouble(3,
o.valorTotal);
      ps.setBoolean(4, o.possuiDanosOcultos); ps.setInt(5, o.sinistro.getId()); ps.setString(6,
o.seguradora.getNome());
      return ps.executeUpdate() == 1 ? null : "Orçamento não cadastrado";
    } catch (SQLException e) { e.printStackTrace(); return "Erro ao alterar orçamento no banco de
dados"; }
  }

  public static String removerOrçamento(int idSinistro, String nomeSeguradora) {
```

```
        try (PreparedStatement ps = BD.conexao.prepareStatement("DELETE FROM Orçamentos
WHERE sinistro = ? AND seguradora = ?")) {
            ps.setInt(1, idSinistro); ps.setString(2, nomeSeguradora);
            return ps.executeUpdate() == 1 ? null : "Orçamento não cadastrado";
        } catch (SQLException e) { e.printStackTrace(); return "Erro ao remover orçamento do banco
de dados"; }
    }

    /** Pesquisa orçamento e entidades relacionadas com um filtro específico por entidade. */
    public static ArrayList<Orçamento> pesquisarOrçamentos(Timestamp dataMinima,
            Peça.MarcaPeça marca, Boolean atendimento24h, Sinistro.GrauMonta grauMonta,
            Integer prazoGarantiaMinimo, String corAcessório, boolean todosObjetos) {
        StringBuilder sql = new StringBuilder("SELECT DISTINCT o.data, o.sinistro, o.seguradora "
                + "FROM Orçamentos o JOIN sinistros s ON s.id=o.sinistro "
                + "JOIN seguradoras g ON g.nome=o.seguradora WHERE 1=1");
        if (dataMinima != null) sql.append(" AND o.data >= ?");
        if (atendimento24h != null) sql.append(" AND g.possui_atendimento_24h = ?");
        if (grauMonta != null) sql.append(" AND s.grau_monta = ?");
        if (marca != null) {
            sql.append(" AND EXISTS (SELECT 1 FROM peças_sinistros ps JOIN peças p ON
p.codigo=ps.peça_codigo WHERE ps.sinistro_id=s.id AND p.marca = ?)");
            if (todosObjetos) sql.append(" AND NOT EXISTS (SELECT 1 FROM peças_sinistros ps
JOIN peças p ON p.codigo=ps.peça_codigo WHERE ps.sinistro_id=s.id AND (p.marca IS NULL
OR p.marca <> ?))");
        }
        if (prazoGarantiaMinimo != null) {
            sql.append(" AND EXISTS (SELECT 1 FROM peças_sinistros ps JOIN peça_mecânica pm
ON pm.codigoId=ps.peça_codigo WHERE ps.sinistro_id=s.id AND pm.prazo_garantia >= ?)");
            if (todosObjetos) sql.append(" AND NOT EXISTS (SELECT 1 FROM peças_sinistros ps
JOIN peça_mecânica pm ON pm.codigoId=ps.peça_codigo WHERE ps.sinistro_id=s.id AND
(pm.prazo_garantia IS NULL OR pm.prazo_garantia < ?))");
        }
        if (corAcessório != null && !corAcessório.trim().isEmpty()) {
            sql.append(" AND EXISTS (SELECT 1 FROM peças_sinistros ps JOIN peça_acessórios pa
ON pa.codigoId=ps.peça_codigo WHERE ps.sinistro_id=s.id AND pa.cor LIKE ?)");
            if (todosObjetos) sql.append(" AND NOT EXISTS (SELECT 1 FROM peças_sinistros ps
JOIN peça_acessórios pa ON pa.codigoId=ps.peça_codigo WHERE ps.sinistro_id=s.id AND
(pa.cor IS NULL OR pa.cor NOT LIKE ?))");
        }
        sql.append(" ORDER BY o.data, o.sinistro, o.seguradora");
```

```
        ArrayList<Orçamento> resultado = new ArrayList<>();
        try (PreparedStatement ps = BD.conexao.prepareStatement(sql.toString())) {
            int i = 0;
            // A janela recebe a data mínima como horário local (meia-noite
            // quando o usuário informa somente AAAA-MM-DD). Enviar como
            // Timestamp permite ao driver aplicar conversão de fuso horário
            // e deslocar esse limite para o dia anterior. O MySQL interpreta
            // esta representação SQL diretamente na comparação com o campo.
            if (dataMinima != null) ps.setString(++i, dataMinima.toString());
            if (atendimento24h != null) ps.setBoolean(++i, atendimento24h);
            if (grauMonta != null) ps.setString(++i, grauMonta.toString());
            if (marca != null) ps.setString(++i, marca.toString());
            if (marca != null && todosObjetos) ps.setString(++i, marca.toString());
            if (prazoGarantiaMinimo != null) ps.setInt(++i, prazoGarantiaMinimo);
            if (prazoGarantiaMinimo != null && todosObjetos) ps.setInt(++i, prazoGarantiaMinimo);
            if (corAcessório != null && !corAcessório.trim().isEmpty()) {
                String padrãoCor = "%" + corAcessório.trim() + "%";
                ps.setString(++i, padrãoCor);
                if (todosObjetos) ps.setString(++i, padrãoCor);
            }
            try (ResultSet rs = ps.executeQuery()) {
                while (rs.next()) {
                    Orçamento orçamento = buscarOrçamento(rs.getInt("sinistro"),
rs.getString("seguradora"));
                    if (orçamento != null) resultado.add(orçamento);
                }
            }
        } catch (SQLException e) { e.printStackTrace(); }
        return resultado;
    }
}
```

$$$ src.entidades.Peça

```
package entidades;

import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Peça {
   public enum MarcaPeça {
      OEM("oem"), ORIGINAL("original"), GENUINA("genuina");
      private final String texto;
      MarcaPeça(String texto) { this.texto = texto; }
      public static MarcaPeça fromTexto(String texto) {
         if (texto == null) throw new IllegalArgumentException("Marca não informada");
         String valor = texto.trim().toLowerCase(Locale.ROOT);
         for (MarcaPeça marca : values()) if (marca.texto.equals(valor)) return marca;
         throw new IllegalArgumentException("Marca inválida: " + texto);
      }
      @Override public String toString() { return texto; }
   }

   protected int codigo;
   protected String nome;
   protected MarcaPeça marca;
   protected double preco;
   protected Integer diasGarantia;
   protected String cor;
   protected boolean sobEncomenda;

   public Peça(int codigo, String nome, MarcaPeça marca, double preco,
         Integer diasGarantia, String cor) {
      this(codigo, nome, marca, preco, diasGarantia, cor, false);
```

```
    }
    public Peça(int codigo, String nome, MarcaPeça marca, double preco,
            Integer diasGarantia, String cor, boolean sobEncomenda) {
        this.codigo=codigo; this.nome=nome; this.marca=marca; this.preco=preco;
        this.diasGarantia=diasGarantia; this.cor=cor; this.sobEncomenda=sobEncomenda;
    }
    public Peça() { this(0,null,null,0.0,null,null); }
    public int getCodigo() { return codigo; }
    public String getNome() { return nome; }
    public MarcaPeça getMarca() { return marca; }
    public MarcaPeça getCategoria() { return marca; }
    public double getPreco() { return preco; }
    public Integer getDiasGarantia() { return diasGarantia; }
    public String getCor() { return cor; }
    public boolean isSobEncomenda() { return sobEncomenda; }
    public void setCodigo(int v) { codigo=v; }
    public void setNome(String v) { nome=v; }
    public void setMarca(MarcaPeça v) { marca=v; }
    public void setCategoria(MarcaPeça v) { marca=v; }
    public void setPreco(double v) { preco=v; }
    public void setDiasGarantia(Integer v) { diasGarantia=v; }
    public void setCor(String v) { cor=v; }
    public void setSobEncomenda(boolean v) { sobEncomenda=v; }
    public Peça getVisao() { return this; }
    @Override public String toString() {
        String subtipo = this instanceof PeçaMecânica ? "Peça Mecânica"
                : this instanceof PeçaAcessório ? "Peça Acessórios" : "Peça";
        return "[" + codigo + "] " + subtipo + ": " + nome + " [" + marca + "]";
    }

    static String colunas() {
        return
"p.codigo,p.nome,p.marca,p.`preço`,p.sob_encomenda,m.categoria_mecânica,m.prazo_garantia,m.
mecanico_proprio,a.categoria_acessorio,a.cor,a.funileiro_proprio";
    }
    static Peça criarVisao(ResultSet rs) throws SQLException {
        int id=rs.getInt("codigo"); String nome=rs.getString("nome");
```

```
        MarcaPeça marca=MarcaPeça.fromTexto(rs.getString("marca")); double
preco=rs.getDouble("preço");
        boolean sobEncomenda=rs.getBoolean("sob_encomenda");
        String catMec=rs.getString("categoria_mecânica");
        if (catMec != null) return new PeçaMecânica(id,nome,marca,preco,
                PeçaMecânica.CategoriaMecânica.fromTexto(catMec),
(Integer)rs.getObject("prazo_garantia"),rs.getBoolean("mecanico_proprio"),sobEncomenda);
        String catAcessorio=rs.getString("categoria_acessorio");
        if (catAcessorio != null) return new PeçaAcessório(id,nome,marca,preco,

PeçaAcessório.CategoriaAcessório.fromTexto(catAcessorio),rs.getString("cor"),rs.getBoolean("funi
leiro_proprio"),sobEncomenda);
        return new Peça(id,nome,marca,preco,null,null,sobEncomenda);
    }
    static String consultaBase() {
        return " FROM peças p LEFT JOIN peça_mecânica m ON m.codigoId=p.codigo LEFT JOIN
peça_acessórios a ON a.codigoId=p.codigo";
    }
    public static Peça[] getVisoes() {
        ArrayList<Peça> lista=new ArrayList<>();
        try(PreparedStatement ps=BD.conexao.prepareStatement("SELECT "+colunas()
+consultaBase()+" ORDER BY p.codigo"); ResultSet rs=ps.executeQuery()) {
            while(rs.next()) lista.add(criarVisao(rs));
        } catch(SQLException|IllegalArgumentException e) { e.printStackTrace(); }
        return lista.toArray(new Peça[0]);
    }
    public static Peça buscarPeças(int codigo) { return buscarPor("p.codigo=?", codigo); }
    private static Peça buscarPor(String filtro,int valor) {
        try(PreparedStatement ps=BD.conexao.prepareStatement("SELECT "+colunas()
+consultaBase()+" WHERE "+filtro)) {
            ps.setInt(1,valor); try(ResultSet rs=ps.executeQuery()) { return rs.next()?criarVisao(rs):null;
}
        } catch(SQLException|IllegalArgumentException e) { e.printStackTrace(); return null; }
    }
    public static Peça buscarPeças(String nome) {
        try(PreparedStatement ps=BD.conexao.prepareStatement("SELECT "+colunas()
+consultaBase()+" WHERE p.nome=?")) {
            ps.setString(1,nome); try(ResultSet rs=ps.executeQuery()) { return rs.next()?
criarVisao(rs):null; }
        } catch(SQLException|IllegalArgumentException e) { e.printStackTrace(); return null; }
```

```
    }
    public static Peça[] buscarPeçasPorSinistro(int id) { return
PeçaSinistro.buscarPeçasPorSinistro(id); }
    public String inserir() {
        try(PreparedStatement ps=BD.conexao.prepareStatement("INSERT INTO
peças(codigo,nome,marca,`preço`,sob_encomenda) VALUES(?,?,?,?,?)")) {
            ps.setInt(1,codigo); ps.setString(2,nome); ps.setString(3,marca.toString());
ps.setDouble(4,preco); ps.setBoolean(5,sobEncomenda); ps.executeUpdate(); return null;
        } catch(SQLException|NullPointerException e) { e.printStackTrace(); return "Erro ao inserir
peça"; }
    }
    public static String inserirPeças(Peça p) { return p.inserir(); }
    public String alterar() {
        try(PreparedStatement ps=BD.conexao.prepareStatement("UPDATE peças SET
nome=?,marca=?,`preço`=?,sob_encomenda=? WHERE codigo=?")) {
            ps.setString(1,nome); ps.setString(2,marca.toString()); ps.setDouble(3,preco);
ps.setBoolean(4,sobEncomenda); ps.setInt(5,codigo); ps.executeUpdate(); return null;
        } catch(SQLException|NullPointerException e) { e.printStackTrace(); return "Erro ao alterar
peça"; }
    }
    public static String alterarPeças(Peça p) { return p.alterar(); }
    public static String removerPeças(int codigo) {
        return removerPeçasPor("codigo", codigo);
    }
    public static String removerPeças(String nome) {
        return removerPeçasPor("nome", nome);
    }

    /**
     * Remove uma peça e todos os seus vínculos em uma única transação.
     * A remoção explícita do lado dependente mantém o código correto mesmo
     * quando o banco foi criado com uma versão antiga do esquema, sem
     * ON DELETE CASCADE.
     */
    private static String removerPeçasPor(String coluna, Object valor) {
        Connection conexao = BD.conexao;
        boolean autoCommitAnterior = true;
        boolean transacaoIniciada = false;
        ArrayList<Integer> codigos = new ArrayList<>();
```

```
try {
    autoCommitAnterior = conexao.getAutoCommit();
    conexao.setAutoCommit(false);
    transacaoIniciada = true;

    try (PreparedStatement consulta = conexao.prepareStatement(
            "SELECT codigo FROM peças WHERE " + coluna + " = ?")) {
        consulta.setObject(1, valor);
        try (ResultSet resultados = consulta.executeQuery()) {
            while (resultados.next()) codigos.add(resultados.getInt("codigo"));
        }
    }

    if (codigos.isEmpty()) {
        conexao.rollback();
        return "Peça não cadastrada";
    }

    for (int codigo : codigos) {
        try (PreparedStatement removerVinculos = conexao.prepareStatement(
                "DELETE FROM peças_sinistros WHERE peça_codigo = ?")) {
            removerVinculos.setInt(1, codigo);
            removerVinculos.executeUpdate();
        }
        try (PreparedStatement removerMecanica = conexao.prepareStatement(
                "DELETE FROM `peça_mecânica` WHERE codigoId = ?")) {
            removerMecanica.setInt(1, codigo);
            removerMecanica.executeUpdate();
        }
        try (PreparedStatement removerAcessorios = conexao.prepareStatement(
                "DELETE FROM `peça_acessórios` WHERE codigoId = ?")) {
            removerAcessorios.setInt(1, codigo);
            removerAcessorios.executeUpdate();
        }
        try (PreparedStatement removerPeca = conexao.prepareStatement(
                "DELETE FROM peças WHERE codigo = ?")) {
```

```
                removerPeca.setInt(1, codigo);
                if (removerPeca.executeUpdate() != 1) {
                    throw new SQLException("A peça " + codigo + " não foi removida");
                }
            }
        }

        conexao.commit();
        return null;
    } catch (SQLException | RuntimeException e) {
        if (transacaoIniciada) {
            try {
                conexao.rollback();
            } catch (SQLException rollback) {
                e.addSuppressed(rollback);
            }
        }
        e.printStackTrace();
        return "Erro ao remover peça";
    } finally {
        if (transacaoIniciada) {
            try {
                conexao.setAutoCommit(autoCommitAnterior);
            } catch (SQLException restauraAutoCommit) {
                restauraAutoCommit.printStackTrace();
            }
        }
    }
  }
}
```

$$$ src.entidades.PeçaAcessório

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Locale;
import persistência.BD;

public class PeçaAcessório extends Peça {
   public enum CategoriaAcessório {
      CARROCERIA("carroceria"), FAROL_LANTERNA("farol_lanterna"),
      PARABRISAS_VIGIA("parabrisas_vigia"), PAINEL("painel"), PEDAL("pedal"),
      ASSENTO("assento"), ASSOALHO("assoalho");
      private final String texto;
      CategoriaAcessório(String texto) { this.texto = texto; }
      public static CategoriaAcessório fromTexto(String texto) {
         if (texto == null) return null;
         String normalizado = texto.trim().toLowerCase(Locale.ROOT);
         for (CategoriaAcessório valor : values()) if (valor.texto.equals(normalizado)) return valor;
         throw new IllegalArgumentException("Categoria do acessório inválida: " + texto);
      }
      @Override public String toString() { return texto; }
   }

   private CategoriaAcessório categoriaAcessório;
   private String cor;
   private boolean funileiroProprio;

   public PeçaAcessório(int codigo, String nome, MarcaPeça marca, double preco,
         CategoriaAcessório categoriaAcessório, String cor, boolean funileiroProprio) {
      this(codigo, nome, marca, preco, categoriaAcessório, cor, funileiroProprio, false);
   }

   public PeçaAcessório(int codigo, String nome, MarcaPeça marca, double preco,
         CategoriaAcessório categoriaAcessório, String cor, boolean funileiroProprio, boolean
```

```
sobEncomenda) {
        super(codigo, nome, marca, preco, null, cor);
        setSobEncomenda(sobEncomenda);
        this.categoriaAcessório = categoriaAcessório;
        this.cor = cor;
        this.funileiroProprio = funileiroProprio;
    }

    public PeçaAcessório(int codigo, String nome, MarcaPeça marca, double preco,
            String categoriaAcessório, String cor) {
        this(codigo, nome, marca, preco, CategoriaAcessório.fromTexto(categoriaAcessório), cor,
false);
    }

    public PeçaAcessório() { this(0, null, null, 0.0, (CategoriaAcessório) null, null, false); }
    public CategoriaAcessório getCategoriaAcessório() { return categoriaAcessório; }
    @Override public String getCor() { return cor; }
    public boolean isFunileiroProprio() { return funileiroProprio; }
    public void setCategoriaAcessório(CategoriaAcessório valor) { categoriaAcessório = valor; }
    @Override public void setCor(String valor) { cor = valor; super.setCor(valor); }
    public void setFunileiroProprio(boolean valor) { funileiroProprio = valor; }

    public static PeçaAcessório buscar(int codigo) {
        String sql = "SELECT
p.codigo,p.nome,p.marca,p.`preço`,p.sob_encomenda,a.categoria_acessorio,a.cor,a.funileiro_propri
o FROM peças p JOIN peça_acessórios a ON a.codigoId=p.codigo WHERE p.codigo=?";
        try (PreparedStatement ps = BD.conexao.prepareStatement(sql)) {
            ps.setInt(1, codigo);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return null;
                return new PeçaAcessório(rs.getInt("codigo"), rs.getString("nome"),
                        MarcaPeça.fromTexto(rs.getString("marca")), rs.getDouble("preço"),
                        CategoriaAcessório.fromTexto(rs.getString("categoria_acessorio")),
                        rs.getString("cor"), rs.getBoolean("funileiro_proprio"),
rs.getBoolean("sob_encomenda"));
            }
        } catch (SQLException e) { e.printStackTrace(); return null; }
    }
```

```
   @Override public String inserir() {
      boolean autoCommit;
      try {
         autoCommit = BD.conexao.getAutoCommit(); BD.conexao.setAutoCommit(false);
         String erro = super.inserir(); if (erro != null) throw new SQLException(erro);
         try (PreparedStatement sub = BD.conexao.prepareStatement(
               "INSERT INTO peça_acessórios(categoria_acessorio,cor,funileiro_proprio,codigoId)
VALUES(?,?,?,?)")) {
            sub.setString(1, categoriaAcessório.toString()); sub.setString(2, cor);
            sub.setBoolean(3, funileiroProprio); sub.setInt(4, codigo); sub.executeUpdate();
         }
         BD.conexao.commit(); BD.conexao.setAutoCommit(autoCommit); return null;
      } catch (SQLException | NullPointerException e) {
         try { BD.conexao.rollback(); BD.conexao.setAutoCommit(true); } catch (SQLException
rollback) { e.addSuppressed(rollback); }
         e.printStackTrace(); return "Erro ao inserir peça de acessórios";
      }
   }


   @Override public String alterar() {
      String erro = super.alterar(); if (erro != null) return erro;
      try (PreparedStatement ps = BD.conexao.prepareStatement(
            "INSERT INTO peça_acessórios(categoria_acessorio,cor,funileiro_proprio,codigoId)
VALUES(?,?,?,?) "
            + "ON DUPLICATE KEY UPDATE
categoria_acessorio=VALUES(categoria_acessorio),cor=VALUES(cor),funileiro_proprio=VALUE
S(funileiro_proprio)")) {
         ps.setString(1, categoriaAcessório.toString()); ps.setString(2, cor);
         ps.setBoolean(3, funileiroProprio); ps.setInt(4, codigo); ps.executeUpdate();
         try (PreparedStatement remover = BD.conexao.prepareStatement("DELETE FROM
peça_mecânica WHERE codigoId=?")) {
            remover.setInt(1, codigo); remover.executeUpdate();
         }
         return null;
      } catch (SQLException | NullPointerException e) { e.printStackTrace(); return "Erro ao alterar
peça de acessórios"; }
   }
}
```

$$$ src.entidades.PeçaMecânica

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.Locale;
import persistência.BD;

public class PeçaMecânica extends Peça {
   public enum CategoriaMecânica {
      MOTOR("motor"), TRANSMISSÃO("transmissão"), ARREFECIMENTO("arrefecimento"),
      SUSPENSÃO("suspensão"), DIREÇÃO("direção");
      private final String texto;
      CategoriaMecânica(String texto) { this.texto = texto; }
      public static CategoriaMecânica fromTexto(String texto) {
         if (texto == null) return null;
         String normalizado = texto.trim().toLowerCase(Locale.ROOT);
         for (CategoriaMecânica valor : values()) if (valor.texto.equals(normalizado)) return valor;
         throw new IllegalArgumentException("Categoria mecânica inválida: " + texto);
      }
      @Override public String toString() { return texto; }
   }

   private CategoriaMecânica categoriaMecânica;
   private Integer prazoGarantia;
   private boolean mecanicoProprio;

   public PeçaMecânica(int codigo, String nome, MarcaPeça marca, double preco,
         CategoriaMecânica categoriaMecânica, Integer prazoGarantia, boolean mecanicoProprio) {
      this(codigo, nome, marca, preco, categoriaMecânica, prazoGarantia, mecanicoProprio, false);
   }

   public PeçaMecânica(int codigo, String nome, MarcaPeça marca, double preco,
         CategoriaMecânica categoriaMecânica, Integer prazoGarantia, boolean mecanicoProprio,
boolean sobEncomenda) {
```

```
        super(codigo, nome, marca, preco, prazoGarantia, null);
        setSobEncomenda(sobEncomenda);
        this.categoriaMecânica = categoriaMecânica;
        this.prazoGarantia = prazoGarantia;
        this.mecanicoProprio = mecanicoProprio;
    }

    public PeçaMecânica(int codigo, String nome, MarcaPeça marca, double preco,
            String categoriaMecânica, Integer prazoGarantia) {
        this(codigo, nome, marca, preco, CategoriaMecânica.fromTexto(categoriaMecânica),
prazoGarantia, false);
    }

    public PeçaMecânica() { this(0, null, null, 0.0, (CategoriaMecânica) null, null, false); }
    public CategoriaMecânica getCategoriaMecânica() { return categoriaMecânica; }
    public Integer getPrazoGarantia() { return prazoGarantia; }
    public boolean isMecanicoProprio() { return mecanicoProprio; }
    public void setCategoriaMecânica(CategoriaMecânica valor) { categoriaMecânica = valor; }
    public void setPrazoGarantia(Integer valor) { prazoGarantia = valor; }
    public void setMecanicoProprio(boolean valor) { mecanicoProprio = valor; }

    public static PeçaMecânica buscar(int codigo) {
        String sql = "SELECT
p.codigo,p.nome,p.marca,p.`preço`,p.sob_encomenda,m.categoria_mecânica,m.prazo_garantia,m.m
ecanico_proprio FROM peças p JOIN peça_mecânica m ON m.codigoId=p.codigo WHERE
p.codigo=?";
        try (PreparedStatement ps = BD.conexao.prepareStatement(sql)) {
            ps.setInt(1, codigo);
            try (ResultSet rs = ps.executeQuery()) {
                if (!rs.next()) return null;
                return new PeçaMecânica(rs.getInt("codigo"), rs.getString("nome"),
                        MarcaPeça.fromTexto(rs.getString("marca")), rs.getDouble("preço"),
                        CategoriaMecânica.fromTexto(rs.getString("categoria_mecânica")),
                        (Integer) rs.getObject("prazo_garantia"), rs.getBoolean("mecanico_proprio"),
rs.getBoolean("sob_encomenda"));
            }
        } catch (SQLException e) { e.printStackTrace(); return null; }
    }
```

```
    @Override public String inserir() {
        boolean autoCommit;
        try {
            autoCommit = BD.conexao.getAutoCommit();
            BD.conexao.setAutoCommit(false);
            String erro = super.inserir();
            if (erro != null) throw new SQLException(erro);
            try (PreparedStatement sub = BD.conexao.prepareStatement(
                    "INSERT INTO
peça_mecânica(categoria_mecânica,prazo_garantia,mecanico_proprio,codigoId)
VALUES(?,?,?,?)")) {
                sub.setString(1, categoriaMecânica.toString());
                if (prazoGarantia == null) sub.setNull(2, java.sql.Types.INTEGER); else sub.setInt(2,
prazoGarantia);
                sub.setBoolean(3, mecanicoProprio); sub.setInt(4, codigo); sub.executeUpdate();
            }
            BD.conexao.commit(); BD.conexao.setAutoCommit(autoCommit); return null;
        } catch (SQLException | NullPointerException e) {
            try { BD.conexao.rollback(); BD.conexao.setAutoCommit(true); } catch (SQLException
rollback) { e.addSuppressed(rollback); }
            e.printStackTrace(); return "Erro ao inserir peça mecânica";
        }
    }


    @Override public String alterar() {
        String erro = super.alterar(); if (erro != null) return erro;
        try (PreparedStatement ps = BD.conexao.prepareStatement(
                "INSERT INTO
peça_mecânica(categoria_mecânica,prazo_garantia,mecanico_proprio,codigoId) VALUES(?,?,?,?)
"
                + "ON DUPLICATE KEY UPDATE
categoria_mecânica=VALUES(categoria_mecânica),prazo_garantia=VALUES(prazo_garantia),me
canico_proprio=VALUES(mecanico_proprio)")) {
            ps.setString(1, categoriaMecânica.toString());
            if (prazoGarantia == null) ps.setNull(2, java.sql.Types.INTEGER); else ps.setInt(2,
prazoGarantia);
            ps.setBoolean(3, mecanicoProprio); ps.setInt(4, codigo); ps.executeUpdate();
            try (PreparedStatement remover = BD.conexao.prepareStatement("DELETE FROM
peça_acessórios WHERE codigoId=?")) {
```

```
            remover.setInt(1, codigo); remover.executeUpdate();
        }
        return null;
    } catch (SQLException | NullPointerException e) { e.printStackTrace(); return "Erro ao alterar peça mecânica"; }
  }
}
```

$$$ src.entidades.PeçaSinistro

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import persistência.BD;

public class PeçaSinistro {

   private final int pecaCodigo;
   private final int sinistroId;

   public PeçaSinistro(int pecaCodigo, int sinistroId) {
      this.pecaCodigo = pecaCodigo;
      this.sinistroId = sinistroId;
   }

   public PeçaSinistro(Peça peca, Sinistro sinistro) {
      this(peca != null ? peca.getCodigo() : 0,
            sinistro != null ? sinistro.getId() : 0);
   }

   public int getPeçaCodigo() {
      return pecaCodigo;
   }

   public int getSinistroId() {
      return sinistroId;
   }

   public String toString() {
      return pecaCodigo + " / " + sinistroId;
   }
```

```
    public static Peça[] buscarPeçasPorSinistro(int sinistroId) {
        ArrayList<Peça> visoes = new ArrayList<>();
        String sql = "SELECT " + Peça.colunas() + Peça.consultaBase()
                + " JOIN peças_sinistros ps ON ps.peça_codigo=p.codigo"
                + " WHERE ps.sinistro_id = ?";

        try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
            comando.setInt(1, sinistroId);
            try (ResultSet resultados = comando.executeQuery()) {
                while (resultados.next()) {
                    try {
                        visoes.add(Peça.criarVisao(resultados));
                    } catch (IllegalArgumentException excecaoEnum) {
                        // Ignora peças com dados fora do domínio esperado.
                    }
                }
            }
        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
        }

        return visoes.toArray(new Peça[0]);
    }

    public static boolean existePeçasSinistros(int pecaCodigo, int sinistroId) {
        String sql = "SELECT COUNT(*) FROM peças_sinistros WHERE peça_codigo = ? AND
sinistro_id = ?";

        try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
            comando.setInt(1, pecaCodigo);
            comando.setInt(2, sinistroId);
            try (ResultSet resultados = comando.executeQuery()) {
                return resultados.next() && resultados.getInt(1) > 0;
            }
        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            return false;
```

```
        }
    }

    public static String inserirPeçasSinistros(Peça peca, Sinistro sinistro) {
        if (peca == null || sinistro == null) {
            return "Peça ou sinistro nao informado";
        }

        String sql = "INSERT INTO peças_sinistros (peça_codigo, sinistro_id) VALUES (?, ?)";

        try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
            comando.setInt(1, peca.getCodigo());
            comando.setInt(2, sinistro.getId());
            comando.executeUpdate();
            return null;
        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            return "Erro na insercao da associacao peca/sinistro no BD";
        }
    }

    public static String removerPeçasSinistros(Peça peca, Sinistro sinistro) {
        if (peca == null || sinistro == null) {
            return "Peça ou sinistro nao informado";
        }

        String sql = "DELETE FROM peças_sinistros WHERE peça_codigo = ? AND sinistro_id
= ?";

        try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
            comando.setInt(1, peca.getCodigo());
            comando.setInt(2, sinistro.getId());
            comando.executeUpdate();
            return null;
        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            return "Erro na remocao da associacao peca/sinistro no BD";
```

```
        }
    }
}
```

$$$ src.entidades.Seguradora

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Seguradora {

    public enum FormaPagamentoPreferencial {
        BOLETO("boleto"),
        CARTAO("cartao"),
        PIX("pix"),
        DEBITO_AUTOMATICO("debito_automatico");

        private final String texto;

        FormaPagamentoPreferencial(String texto) {
            this.texto = texto;
        }

        public static FormaPagamentoPreferencial fromTexto(String texto) {
            if (texto == null) {
                throw new IllegalArgumentException("Forma de pagamento não informada");
            }

            String normalizado = texto.trim().toLowerCase(Locale.ROOT);

            if (normalizado.equals("boleto")) return BOLETO;
            if (normalizado.equals("cartao") || normalizado.equals("cartão")) return CARTAO;
            if (normalizado.equals("pix")) return PIX;
            if (normalizado.equals("debito_automatico")
                    || normalizado.equals("debito automatico")
```

```
                || normalizado.equals("débito_automático")
                || normalizado.equals("débito automático")) {
            return DEBITO_AUTOMATICO;
        }

        throw new IllegalArgumentException("Forma de pagamento inválida: " + texto);
    }

    @Override
    public String toString() {
        return texto;
    }
}

private String nome;
private String cidade;
private double coberturaPercentual;
private boolean possuiAtendimento24h;
private FormaPagamentoPreferencial formaPagamentoPreferencial;

public String getNome() { return nome; }
public String getCidade() { return cidade; }
public double getCoberturaPercentual() { return coberturaPercentual; }
public boolean getPossuiAtendimento24h() { return possuiAtendimento24h; }
public boolean isPossuiAtendimento24h() { return possuiAtendimento24h; }
public FormaPagamentoPreferencial getFormaPagamentoPreferencial() { return
formaPagamentoPreferencial; }

public void setCidade(String cidade) { this.cidade = cidade; }
public void setCoberturaPercentual(double coberturaPercentual) { this.coberturaPercentual =
coberturaPercentual; }
public void setPossuiAtendimento24h(boolean possuiAtendimento24h)
{ this.possuiAtendimento24h = possuiAtendimento24h; }
public void setFormaPagamentoPreferencial(FormaPagamentoPreferencial
formaPagamentoPreferencial) {
    this.formaPagamentoPreferencial = formaPagamentoPreferencial;
}
```

```
public Seguradora(
        String nome,
        String cidade,
        double coberturaPercentual,
        boolean possuiAtendimento24h,
        FormaPagamentoPreferencial formaPagamentoPreferencial
) {
    this.nome = nome;
    this.cidade = cidade;
    this.coberturaPercentual = coberturaPercentual;
    this.possuiAtendimento24h = possuiAtendimento24h;
    this.formaPagamentoPreferencial = formaPagamentoPreferencial;
}

public Seguradora(String nome, String cidade, double coberturaPercentual) {
    this(nome, cidade, coberturaPercentual, false, FormaPagamentoPreferencial.BOLETO);
}

public Seguradora(String nome, String cidade) {
    this(nome, cidade, 0, false, FormaPagamentoPreferencial.BOLETO);
}

public String toString() {
    return nome + " [" + cidade + "]";
}

public Seguradora getVisao() {
    return new Seguradora(
            nome,
            cidade,
            coberturaPercentual,
            possuiAtendimento24h,
            formaPagamentoPreferencial
    );
}

public static Seguradora[] getVisoes() {
```

```
        String sql = "SELECT nome, cidade, cobertura_percentual, possui_atendimento_24h,
forma_pagamento_preferencial FROM seguradoras";
        ResultSet listaResultados = null;
        ArrayList<Seguradora> visoes = new ArrayList<>();

        try {
            PreparedStatement comando = BD.conexao.prepareStatement(sql);
            listaResultados = comando.executeQuery();

            while (listaResultados.next()) {
                String nome = listaResultados.getString("nome");
                String cidade = listaResultados.getString("cidade");
                double coberturaPercentual = listaResultados.getDouble("cobertura_percentual");
                boolean possuiAtendimento24h =
listaResultados.getBoolean("possui_atendimento_24h");
                FormaPagamentoPreferencial formaPagamentoPreferencial =
                        FormaPagamentoPreferencial.fromTexto(
                                listaResultados.getString("forma_pagamento_preferencial")
                        );
                visoes.add(new Seguradora(
                        nome,
                        cidade,
                        coberturaPercentual,
                        possuiAtendimento24h,
                        formaPagamentoPreferencial
                ));
            }

            listaResultados.close();
            comando.close();

        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
        }

        return visoes.toArray(new Seguradora[visoes.size()]);
    }
```

```
    public static Seguradora buscarSeguradora(String nome) {
        String sql = "SELECT * FROM seguradoras WHERE nome = ?";
        ResultSet listaResultados = null;
        Seguradora seguradora = null;

        try {
            PreparedStatement comando = BD.conexao.prepareStatement(sql);
            comando.setString(1, nome);
            listaResultados = comando.executeQuery();

            while (listaResultados.next()) {
                seguradora = new Seguradora(
                        nome,
                        listaResultados.getString("cidade"),
                        listaResultados.getDouble("cobertura_percentual"),
                        listaResultados.getBoolean("possui_atendimento_24h"),
                        FormaPagamentoPreferencial.fromTexto(
                                listaResultados.getString("forma_pagamento_preferencial")
                        )
                );
            }

            listaResultados.close();
            comando.close();

        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            seguradora = null;
        }

        return seguradora;
    }

    public static String inserirSeguradora(Seguradora seguradora) {
        String sql = "INSERT INTO seguradoras (nome, cidade, cobertura_percentual,
possui_atendimento_24h, forma_pagamento_preferencial) VALUES (?,?,?,?,?)";
```

```
        try {
            PreparedStatement comando = BD.conexao.prepareStatement(sql);

            comando.setString(1, seguradora.getNome());
            comando.setString(2, seguradora.getCidade());
            comando.setDouble(3, seguradora.getCoberturaPercentual());
            comando.setBoolean(4, seguradora.getPossuiAtendimento24h());
            comando.setString(5, seguradora.getFormaPagamentoPreferencial().toString());

            comando.executeUpdate();
            comando.close();

            return null;

        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            return "Erro na Inserção da Seguradora no BD";
        }
    }

    public static String alterarSeguradora(Seguradora seguradora) {
        String sql = "UPDATE seguradoras SET cidade = ?, cobertura_percentual = ?,
possui_atendimento_24h = ?, forma_pagamento_preferencial = ?"
                + " WHERE nome = ?";

        try {
            PreparedStatement comando = BD.conexao.prepareStatement(sql);

            comando.setString(1, seguradora.getCidade());
            comando.setDouble(2, seguradora.getCoberturaPercentual());
            comando.setBoolean(3, seguradora.getPossuiAtendimento24h());
            comando.setString(4, seguradora.getFormaPagamentoPreferencial().toString());
            comando.setString(5, seguradora.getNome());

            comando.executeUpdate();
            comando.close();
```

```
            return null;

        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            return "Erro na Alteração da Seguradora no BD";
        }
    }

    public static String removerSeguradora(String nome) {
        String sql = "DELETE FROM seguradoras WHERE nome = ?";

        try {
            PreparedStatement comando = BD.conexao.prepareStatement(sql);

            comando.setString(1, nome);
            comando.executeUpdate();
            comando.close();

            return null;

        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
            return "Erro na Remoção da Seguradora no BD";
        }
    }
}
```

$$$ src.entidades.Sinistro

```
package entidades;

import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.Locale;
import persistência.BD;

public class Sinistro {

   public enum GrauMonta {
      PEQUENA("pequena"), MEDIA("media"), GRANDE("grande");
      private final String texto;
      GrauMonta(String texto) { this.texto = texto; }
      public static GrauMonta fromTexto(String texto) {
         if (texto == null) throw new IllegalArgumentException("Grau de monta não informado");
         String normalizado = texto.trim().toLowerCase(Locale.ROOT)
               .replace("á", "a").replace("à", "a").replace("â", "a")
               .replace("ã", "a").replace("é", "e").replace("ê", "e")
               .replace("í", "i").replace("ó", "o").replace("ô", "o")
               .replace("õ", "o").replace("ú", "u").replace("ç", "c");
         if (normalizado.contains("pequena")) return PEQUENA;
         if (normalizado.contains("media")) return MEDIA;
         if (normalizado.contains("grande")) return GRANDE;
         throw new IllegalArgumentException("Grau de monta inválido: " + texto);
      }
      @Override public String toString() { return texto; }
   }

   private int id;
   private String segurado;
   private String telefone;
   private String cidade;
```

```
private GrauMonta grauMonta;
private boolean perdaTotal;
private ArrayList<Peça> pecas = new ArrayList<Peça>();

public Sinistro(int id, String segurado, String telefone, String cidade,
        GrauMonta grauMonta, boolean perdaTotal) {
    this.id = id;
    this.segurado = segurado;
    this.telefone = telefone;
    this.cidade = cidade;
    this.grauMonta = grauMonta;
    this.perdaTotal = perdaTotal;
}

public Sinistro(String segurado, String telefone, String cidade,
        GrauMonta grauMonta, boolean perdaTotal) {
    this(0, segurado, telefone, cidade, grauMonta, perdaTotal);
}

public int getId() { return id; }
public String getSegurado() { return segurado; }
public String getTelefone() { return telefone; }
public String getCidade() { return cidade; }
public GrauMonta getGrauMonta() { return grauMonta; }
public boolean getPerdaTotal() { return perdaTotal; }
public boolean isPerdaTotal() { return perdaTotal; }
public Peça[] getPeças() { return pecas.toArray(new Peça[0]); }
public void setSegurado(String segurado) { this.segurado = segurado; }
public void setTelefone(String telefone) { this.telefone = telefone; }
public void setCidade(String cidade) { this.cidade = cidade; }
public void setGrauMonta(GrauMonta grauMonta) { this.grauMonta = grauMonta; }
public void setPerdaTotal(boolean perdaTotal) { this.perdaTotal = perdaTotal; }
public void setPeças(Peça[] pecas) {
    this.pecas = new ArrayList<Peça>();
    if (pecas != null) for (Peça peca : pecas) if (peca != null) this.pecas.add(peca);
}
```

```
    @Override
    public String toString() {
        return "#" + id + " - " + segurado + " - " + (cidade != null ? cidade : "")
                + " (" + grauMonta + ")";
    }

    private static Sinistro criarVisao(ResultSet resultado) throws SQLException {
        return new Sinistro(resultado.getInt("id"), resultado.getString("segurado"),
                resultado.getString("telefone"), resultado.getString("cidade"),
                GrauMonta.fromTexto(resultado.getString("grau_monta")),
                resultado.getBoolean("perda_total"));
    }

    public static Sinistro[] getVisoes() {
        ArrayList<Sinistro> visoes = new ArrayList<Sinistro>();
        String sql = "SELECT id, segurado, telefone, cidade, grau_monta, perda_total FROM sinistros
ORDER BY id";
        try (PreparedStatement comando = BD.conexao.prepareStatement(sql);
                ResultSet resultados = comando.executeQuery()) {
            while (resultados.next()) visoes.add(criarVisao(resultados));
        } catch (SQLException | IllegalArgumentException excecao) { excecao.printStackTrace(); }
        return visoes.toArray(new Sinistro[0]);
    }

    public static Sinistro buscarSinistro(int id) {
        String sql = "SELECT id, segurado, telefone, cidade, grau_monta, perda_total FROM sinistros
WHERE id = ?";
        try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
            comando.setInt(1, id);
            try (ResultSet resultados = comando.executeQuery()) {
                if (!resultados.next()) return null;
                Sinistro sinistro = criarVisao(resultados);
                sinistro.setPeças(PeçaSinistro.buscarPeçasPorSinistro(id));
                return sinistro;
            }
        } catch (SQLException | IllegalArgumentException excecao) {
            excecao.printStackTrace();
```

```
            return null;
        }
    }

    public static String inserirSinistro(Sinistro sinistro) {
        String sql = "INSERT INTO sinistros (segurado, telefone, cidade, grau_monta, perda_total)
VALUES (?, ?, ?, ?, ?)";
        try (PreparedStatement comando = BD.conexao.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
            comando.setString(1, sinistro.getSegurado());
            comando.setString(2, sinistro.getTelefone());
            comando.setString(3, sinistro.getCidade());
            comando.setString(4, sinistro.getGrauMonta().toString());
            comando.setBoolean(5, sinistro.getPerdaTotal());
            comando.executeUpdate();
            try (ResultSet chaves = comando.getGeneratedKeys()) {
                if (chaves.next()) sinistro.id = chaves.getInt(1);
            }
            return sinistro.id > 0 ? null : "Erro ao obter o ID do Sinistro criado";
        } catch (SQLException excecao) {
            excecao.printStackTrace();
            return "Erro na Insercao do Sinistro no BD";
        }
    }

    public static String alterarSinistro(Sinistro sinistro) {
        String sql = "UPDATE sinistros SET segurado = ?, telefone = ?, cidade = ?, grau_monta = ?,
perda_total = ? WHERE id = ?";
        try (PreparedStatement comando = BD.conexao.prepareStatement(sql)) {
            comando.setString(1, sinistro.getSegurado());
            comando.setString(2, sinistro.getTelefone());
            comando.setString(3, sinistro.getCidade());
            comando.setString(4, sinistro.getGrauMonta().toString());
            comando.setBoolean(5, sinistro.getPerdaTotal());
            comando.setInt(6, sinistro.getId());
            return comando.executeUpdate() == 1 ? null : "Sinistro não cadastrado";
        } catch (SQLException excecao) {
            excecao.printStackTrace();
```

```
            return "Erro na Alteracao do Sinistro no BD";
        }
    }

    public static String removerSinistro(int id) {
        try (PreparedStatement comando = BD.conexao.prepareStatement("DELETE FROM sinistros
WHERE id = ?")) {
            comando.setInt(1, id);
            return comando.executeUpdate() == 1 ? null : "Sinistro não cadastrado";
        } catch (SQLException excecao) {
            excecao.printStackTrace();
            return "Erro na Remocao do Sinistro no BD";
        }
    }
}
```

$$$ src.interfaces.JanelaCadastroOrçamentos

```
package interfaces;

import controles.ControladorCadastroOrçamentos;
import entidades.Orçamento;
import entidades.Seguradora;
import entidades.Sinistro;
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.sql.Timestamp;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class JanelaCadastroOrçamentos extends JFrame {
    private final ControladorCadastroOrçamentos controlador;
    private int idSinistroConsultado = -1;
    private String seguradoraConsultada;

    public JanelaCadastroOrçamentos(ControladorCadastroOrçamentos controlador) {
        this.controlador = controlador;
        initComponents();
        setTitle("Cadastrar Orçamentos");
        setResizable(false);

        sinistros.setModel(new DefaultComboBoxModel<>(Sinistro.getVisoes()));
        seguradoras.setModel(new DefaultComboBoxModel<>(Seguradora.getVisoes()));
        orçamentos.setModel(new DefaultComboBoxModel<>(Orçamento.getVisoes()));
```

```
    atualizarLista(-1, null);
    limparCampos();
    pack();
    setSize(590, 460);
    setLocationByPlatform(true);
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
private void initComponents() {
    java.awt.GridBagConstraints c = new java.awt.GridBagConstraints();
    orçamentosLabel = new javax.swing.JLabel();
    sinistrosLabel = new javax.swing.JLabel();
    seguradorasLabel = new javax.swing.JLabel();
    statusLabel = new javax.swing.JLabel();
    dataLabel = new javax.swing.JLabel();
    valorTotalLabel = new javax.swing.JLabel();
    danosOcultosLabel = new javax.swing.JLabel();
    orçamentos = new javax.swing.JComboBox<>();
    sinistros = new javax.swing.JComboBox<>();
    seguradoras = new javax.swing.JComboBox<>();
    status = new javax.swing.JComboBox<>(Orçamento.Status.values());
    data = new javax.swing.JTextField(19);
    data.setEditable(false);
    valorTotal = new javax.swing.JTextField(20);
    java.awt.Dimension tamanhoValorTotal = valorTotal.getPreferredSize();
    tamanhoValorTotal.width = 220;
    valorTotal.setPreferredSize(tamanhoValorTotal);
    valorTotal.setMinimumSize(tamanhoValorTotal);
    danosOcultos = new javax.swing.JCheckBox("Possui danos ocultos");
    comandosPanel = new javax.swing.JPanel();
    inserirButton = new javax.swing.JButton("Inserir");
    consultarButton = new javax.swing.JButton("Consultar");
    alterarButton = new javax.swing.JButton("Alterar");
    removerButton = new javax.swing.JButton("Remover");
    limparButton = new javax.swing.JButton("Limpar");
```

```
        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Cadastrar Orçamentos");
        getContentPane().setLayout(new java.awt.BorderLayout(8, 8));
        getContentPane().setLayout(new java.awt.GridBagLayout());
        c.insets = new java.awt.Insets(4, 6, 4, 6);
        c.anchor = java.awt.GridBagConstraints.WEST;
        adicionarCampoGerado(getContentPane(), c, 0, orçamentosLabel, "Orçamentos cadastrados",
orçamentos);
        adicionarCampoGerado(getContentPane(), c, 1, sinistrosLabel, "Sinistro", sinistros);
        adicionarCampoGerado(getContentPane(), c, 2, seguradorasLabel, "Seguradora", seguradoras);
        adicionarCampoGerado(getContentPane(), c, 3, statusLabel, "Status", status);
        adicionarCampoGerado(getContentPane(), c, 4, dataLabel, "Data e hora", data);
        adicionarCampoGerado(getContentPane(), c, 5, valorTotalLabel, "Valor total (R$)",
valorTotal);
        adicionarCampoGerado(getContentPane(), c, 6, danosOcultosLabel, "Danos ocultos",
danosOcultos);


        comandosPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.CENTER));
        comandosPanel.add(inserirButton);
        comandosPanel.add(consultarButton);
        comandosPanel.add(alterarButton);
        comandosPanel.add(removerButton);
        comandosPanel.add(limparButton);
        c.gridx = 0; c.gridy = 7; c.gridwidth = 2; c.anchor = java.awt.GridBagConstraints.CENTER;
        getContentPane().add(comandosPanel, c);
        inserirButton.addActionListener(e -> inserir());
        consultarButton.addActionListener(e -> consultar());
        alterarButton.addActionListener(e -> alterar());
        removerButton.addActionListener(e -> remover());
        limparButton.addActionListener(e -> limparCampos());
        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void adicionarCampoGerado(java.awt.Container painel, java.awt.GridBagConstraints c,
            int linha, javax.swing.JLabel rotulo, String texto, java.awt.Component campo) {
        adicionarCampoGerado(painel, c, linha, rotulo, texto, campo,
                campo != status && campo != valorTotal);
    }
```

```
    private void adicionarCampoGerado(java.awt.Container painel, java.awt.GridBagConstraints c,
            int linha, javax.swing.JLabel rotulo, String texto, java.awt.Component campo,
            boolean expandir) {
        rotulo.setText(texto);
        c.gridx = 0; c.gridy = linha; c.fill = java.awt.GridBagConstraints.NONE; c.weightx = 0;
        painel.add(rotulo, c);
        // Mantém os campos estáveis quando um novo modelo é carregado ou limpo.
        c.gridx = 1;
        c.fill = expandir ? java.awt.GridBagConstraints.HORIZONTAL :
java.awt.GridBagConstraints.NONE;
        c.weightx = expandir ? 1.0 : 0;
        painel.add(campo, c);
    }

    private Orçamento obterInformado() {
        Sinistro sinistro = (Sinistro) sinistros.getSelectedItem();
        Seguradora seguradora = (Seguradora) this.seguradoras.getSelectedItem();
        Orçamento.Status statusSelecionado = (Orçamento.Status) status.getSelectedItem();
        if (sinistro == null || seguradora == null || statusSelecionado == null)
            throw new IllegalArgumentException("Selecione o sinistro, a seguradora e o status.");
        Timestamp dataInformada = new Timestamp(System.currentTimeMillis());
        data.setText(dataInformada.toString());
        double valor = Double.parseDouble(valorTotal.getText().trim().replace(',', '.'));
        if (valor < 0) throw new IllegalArgumentException("O valor total não pode ser negativo.");
        return new Orçamento(statusSelecionado, dataInformada, sinistro, seguradora, valor,
danosOcultos.isSelected());
    }

    private void inserir() {
        try {
            Orçamento o = obterInformado();
            String mensagem = controlador.inserirOrçamento(o);
            if (mensagem != null) { erro(mensagem); return; }
            atualizarLista(o.getSinistro().getId(), o.getSeguradora().getNome());
            limparCampos();
        } catch (IllegalArgumentException e) {
```

```
            erro("Informe um valor válido e preencha os campos obrigatórios.");
        }
    }

    private void consultar() {
        Orçamento visão = (Orçamento) orçamentos.getSelectedItem();
        if (visão == null) { erro("Selecione um orçamento."); return; }
        Orçamento o = Orçamento.buscarOrçamento(visão.getSinistro().getId(),
visão.getSeguradora().getNome());
        if (o == null) { erro("Orçamento não cadastrado."); return; }
        idSinistroConsultado = o.getSinistro().getId();
        seguradoraConsultada = o.getSeguradora().getNome();
        selecionarSinistro(idSinistroConsultado);
        selecionarSeguradora(seguradoraConsultada);
        status.setSelectedItem(o.getStatus());
        data.setText(o.getData().toString());
        valorTotal.setText(String.valueOf(o.getValorTotal()));
        danosOcultos.setSelected(o.getPossuiDanosOcultos());
    }

    private void alterar() {
        if (idSinistroConsultado < 0 || seguradoraConsultada == null) {
            erro("Consulte um orçamento antes de alterá-lo."); return;
        }
        try {
            Orçamento o = obterInformado();
            String mensagem = controlador.alterarOrçamento(o, idSinistroConsultado,
seguradoraConsultada);
            if (mensagem != null) { erro(mensagem); return; }
            atualizarLista(idSinistroConsultado, seguradoraConsultada);
            limparCampos();
        } catch (IllegalArgumentException e) {
            erro("Informe um valor válido e preencha os campos obrigatórios.");
        }
    }

    private void remover() {
```

```
        Orçamento visão = (Orçamento) orçamentos.getSelectedItem();
        if (visão == null) { erro("Selecione um orçamento."); return; }
        String mensagem = controlador.removerOrçamento(visão.getSinistro().getId(),
visão.getSeguradora().getNome());
        if (mensagem != null) { erro(mensagem); return; }
        atualizarLista(-1, null);
        limparCampos();
    }

    private void atualizarLista(int idSinistroSelecionado, String seguradoraSelecionada) {
        orçamentos.setModel(new DefaultComboBoxModel<>(Orçamento.getVisoes()));
        if (idSinistroSelecionado >= 0 && seguradoraSelecionada != null) {
            for (int i = 0; i < orçamentos.getItemCount(); i++) {
                Orçamento atual = orçamentos.getItemAt(i);
                if (atual.getSinistro().getId() == idSinistroSelecionado
                        && atual.getSeguradora().getNome().equals(seguradoraSelecionada)) {
                    orçamentos.setSelectedIndex(i); return;
                }
            }
        }
        orçamentos.setSelectedIndex(-1);
    }

    private void selecionarSinistro(int id) {
        for (int i = 0; i < sinistros.getItemCount(); i++)
            if (sinistros.getItemAt(i).getId() == id) { sinistros.setSelectedIndex(i); return; }
    }

    private void selecionarSeguradora(String nome) {
        for (int i = 0; i < seguradoras.getItemCount(); i++)
            if (seguradoras.getItemAt(i).getNome().equals(nome)) { seguradoras.setSelectedIndex(i);
return; }
    }

    private void limparCampos() {
        idSinistroConsultado = -1; seguradoraConsultada = null;
        sinistros.setSelectedIndex(-1); seguradoras.setSelectedIndex(-1); status.setSelectedIndex(-1);
```

```
        data.setText(new Timestamp(System.currentTimeMillis()).toString());
        valorTotal.setText(""); danosOcultos.setSelected(false);
    }

    private void erro(String mensagem) {
        JOptionPane.showMessageDialog(this, mensagem, "Erro",
JOptionPane.ERROR_MESSAGE);
    }

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JPanel comandosPanel;
    private javax.swing.JLabel danosOcultosLabel;
    private javax.swing.JLabel dataLabel;
    private javax.swing.JButton inserirButton;
    private javax.swing.JButton consultarButton;
    private javax.swing.JButton alterarButton;
    private javax.swing.JButton removerButton;
    private javax.swing.JButton limparButton;
    private javax.swing.JLabel orçamentosLabel;
    private javax.swing.JLabel seguradorasLabel;
    private javax.swing.JLabel sinistrosLabel;
    private javax.swing.JLabel statusLabel;
    private javax.swing.JLabel valorTotalLabel;
    private javax.swing.JComboBox<Orçamento> orçamentos;
    private javax.swing.JComboBox<Sinistro> sinistros;
    private javax.swing.JComboBox<Seguradora> seguradoras;
    private javax.swing.JComboBox<Orçamento.Status> status;
    private javax.swing.JTextField data;
    private javax.swing.JTextField valorTotal;
    private javax.swing.JCheckBox danosOcultos;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.interfaces.JanelaCadastroPeças

```
package interfaces;

import controles.ControladorCadastroPeças;
import entidades.Peça;
import entidades.PeçaAcessório;
import entidades.PeçaMecânica;
import java.awt.Frame;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.awt.FlowLayout;
import javax.swing.DefaultComboBoxModel;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JCheckBox;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JTabbedPane;
import javax.swing.JTextField;

public class JanelaCadastroPeças extends JFrame {
   private final ControladorCadastroPeças controlador;
   private Peça[] peçasCadastradas;

   public JanelaCadastroPeças(ControladorCadastroPeças controlador) { this(controlador,null); }
   public JanelaCadastroPeças(ControladorCadastroPeças controlador, Frame owner) {
      this.controlador=controlador;
      initComponents();
      especializaçãoTabbedPane.addTab("Peça mecânica", painelMecânica);
      especializaçãoTabbedPane.addTab("Peça de acessórios", painelAcessórios);
      setTitle("Cadastrar Peças");
      setResizable(false);
      atualizarPeçasCadastradas(-1);
```

```
        limparCampos();
        pack();
        setSize(590, 460);
        if (owner == null) {
            setLocationByPlatform(true);
        } else {
            setLocation(
                    owner.getX() + (owner.getWidth() - getWidth()) / 2,
                    owner.getY() + (owner.getHeight() - getHeight()) / 2
            );
        }
        setAlwaysOnTop(true);
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {
        GridBagConstraints c = new GridBagConstraints();
        peçasComboBox = new JComboBox<>();
        codigoField = new JTextField(8);
        nomeField = new JTextField(24);
        marcaComboBox = new JComboBox<>(Peça.MarcaPeça.values());
        precoField = new JTextField(10);
        sobEncomendaCheckBox = new JCheckBox("Sob encomenda");
        painelMecânica = new JPanel(new GridBagLayout());
        painelAcessórios = new JPanel(new GridBagLayout());
        categoriaMecânicaComboBox = new
JComboBox<>(PeçaMecânica.CategoriaMecânica.values());
        prazoGarantiaTextField = new JTextField(10);
        mecanicoProprioCheckBox = new JCheckBox("Sim");
        categoriaAcessórioComboBox = new
JComboBox<>(PeçaAcessório.CategoriaAcessório.values());
        corTextField = new JTextField(10);
        funileiroProprioCheckBox = new JCheckBox("Sim");
        especializaçãoTabbedPane = new JTabbedPane();
        camposComunsPanel = new JPanel(new GridBagLayout());
        abasAlinhadasPanel = new JPanel(new GridBagLayout());
```

```
        comandosPanel = new JPanel(new FlowLayout(FlowLayout.CENTER));
        inserirButton = new JButton("Inserir");
        consultarButton = new JButton("Consultar");
        alterarButton = new JButton("Alterar");
        removerButton = new JButton("Remover");
        limparButton = new JButton("Limpar");
        peçasLabel = new JLabel(); codigoLabel = new JLabel(); nomeLabel = new JLabel();
        marcaLabel = new JLabel(); precoLabel = new JLabel();
        categoriaMecânicaLabel = new JLabel(); prazoGarantiaLabel = new JLabel();
mecanicoProprioLabel = new JLabel();
        categoriaAcessórioLabel = new JLabel(); corLabel = new JLabel(); funileiroProprioLabel =
new JLabel();


        setDefaultCloseOperation(JFrame.DISPOSE_ON_CLOSE);
        setTitle("Cadastrar Peças");
        getContentPane().setLayout(new GridBagLayout());
        c.insets = new Insets(1, 5, 10, 5); c.anchor = GridBagConstraints.WEST;
        adicionarCampoGerado(camposComunsPanel, c, 0, peçasLabel, "Peças cadastradas",
peçasComboBox, false);
        adicionarCampoGerado(camposComunsPanel, c, 1, codigoLabel, "Código", codigoField,
false);
        adicionarCampoGerado(camposComunsPanel, c, 2, nomeLabel, "Nome", nomeField, false);
        adicionarCampoGerado(camposComunsPanel, c, 3, marcaLabel, "Marca", marcaComboBox,
false);
        adicionarCampoGerado(camposComunsPanel, c, 4, precoLabel, "Preço", precoField, false);
        c.gridx = 1; c.gridy = 5; camposComunsPanel.add(sobEncomendaCheckBox, c);
        c.gridx = 0; c.gridy = 0; c.gridwidth = 2; c.fill = GridBagConstraints.HORIZONTAL;
        getContentPane().add(camposComunsPanel, c);
        painelMecânica.setBorder(javax.swing.BorderFactory.createEmptyBorder(12, 12, 12, 12));
        painelAcessórios.setBorder(javax.swing.BorderFactory.createEmptyBorder(12, 12, 12, 12));
        adicionarCampoGerado(painelMecânica, c, 0, categoriaMecânicaLabel, "Categoria mecânica",
categoriaMecânicaComboBox, true);
        adicionarCampoGerado(painelMecânica, c, 1, prazoGarantiaLabel, "Prazo de garantia (dias)",
prazoGarantiaTextField, true);
        adicionarCampoGerado(painelMecânica, c, 2, mecanicoProprioLabel, "Mecânico próprio",
mecanicoProprioCheckBox, true);
        adicionarCampoGerado(painelAcessórios, c, 0, categoriaAcessórioLabel, "Categoria do
acessório", categoriaAcessórioComboBox, true);
        adicionarCampoGerado(painelAcessórios, c, 1, corLabel, "Cor", corTextField, true);
```

```
        adicionarCampoGerado(painelAcessórios, c, 2, funileiroProprioLabel, "Funileiro próprio",
funileiroProprioCheckBox, true);
        abasAlinhadasPanel.setBorder(javax.swing.BorderFactory.createEmptyBorder(8, 8, 8, 8));
        abasAlinhadasPanel.add(especializaçãoTabbedPane);
        c.gridx = 0; c.gridy = 1; c.gridwidth = 2; c.fill = GridBagConstraints.BOTH; c.weightx = 1.0;
c.weighty = 1.0;
        getContentPane().add(abasAlinhadasPanel, c);
        comandosPanel.add(inserirButton); comandosPanel.add(consultarButton);
        comandosPanel.add(alterarButton); comandosPanel.add(removerButton);
comandosPanel.add(limparButton);
        c.gridx = 0; c.gridy = 2; c.gridwidth = 2; c.fill = GridBagConstraints.NONE; c.weightx = 0;
c.weighty = 0;
        getContentPane().add(comandosPanel, c);
        inserirButton.addActionListener(e -> inserir()); consultarButton.addActionListener(e ->
consultar());
        alterarButton.addActionListener(e -> alterar()); removerButton.addActionListener(e ->
remover());
        limparButton.addActionListener(e -> limparCampos());
        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void adicionarCampoGerado(java.awt.Container painel, GridBagConstraints c, int linha,
            JLabel rotulo, String texto, java.awt.Component campo, boolean expandir) {
        rotulo.setText(texto);
        c.gridwidth = 1;
        c.gridx = 0; c.gridy = linha; c.fill = GridBagConstraints.NONE; c.weightx = 0;
        painel.add(rotulo, c);
        c.gridx = 1; c.fill = expandir ? GridBagConstraints.HORIZONTAL :
GridBagConstraints.NONE;
        c.weightx = expandir ? 1.0 : 0;
        painel.add(campo, c);
    }
    private void atualizarPeçasCadastradas(int codigoSelecionado) {
        peçasCadastradas=Peça.getVisoes(); peçasComboBox.setModel(new
DefaultComboBoxModel<>(peçasCadastradas));
        for(Peça p:peçasCadastradas) if(p.getCodigo()==codigoSelecionado)
{ peçasComboBox.setSelectedItem(p);break; }
    }
    private Peça obterPeçaInformada() {
```

```
        int codigo; double preco;
        try { codigo=Integer.parseInt(codigoField.getText().trim()); } catch(NumberFormatException
e) { throw new IllegalArgumentException("Informe um código inteiro válido."); }
        String nome=nomeField.getText().trim(); if(nome.isEmpty()) throw new
IllegalArgumentException("Informe o nome da peça.");
        Peça.MarcaPeça marca=(Peça.MarcaPeça)marcaComboBox.getSelectedItem();
if(marca==null) throw new IllegalArgumentException("Selecione a marca.");
        try { preco=Double.parseDouble(precoField.getText().trim().replace(',','.')); }
catch(NumberFormatException e) { throw new IllegalArgumentException("Informe um preço
válido."); }
        if(preco<0) throw new IllegalArgumentException("O preço não pode ser negativo.");
        try {
            if(especializaçãoTabbedPane.getSelectedIndex()==0) {
                PeçaMecânica.CategoriaMecânica categoria=(PeçaMecânica.CategoriaMecânica)
categoriaMecânicaComboBox.getSelectedItem();
                if(categoria==null) throw new IllegalArgumentException("Selecione a categoria
mecânica.");
                String prazoTexto=prazoGarantiaTextField.getText().trim();
                Integer prazo=prazoTexto.isEmpty()?null:Integer.valueOf(prazoTexto);
                if(prazo!=null && prazo<0) throw new IllegalArgumentException("O prazo de garantia
não pode ser negativo.");
                return new
PeçaMecânica(codigo,nome,marca,preco,categoria,prazo,mecanicoProprioCheckBox.isSelected(),s
obEncomendaCheckBox.isSelected());
            }
            PeçaAcessório.CategoriaAcessório categoria=(PeçaAcessório.CategoriaAcessório)
categoriaAcessórioComboBox.getSelectedItem();
            if(categoria==null) throw new IllegalArgumentException("Selecione a categoria do
acessório.");
            String cor=corTextField.getText().trim();
            return new PeçaAcessório(codigo,nome,marca,preco,categoria,cor.isEmpty()?
null:cor,funileiroProprioCheckBox.isSelected(),sobEncomendaCheckBox.isSelected());
        } catch(NumberFormatException e) { throw new IllegalArgumentException("Informe um
prazo de garantia inteiro válido."); }
    }
    private void inserir() {
        try { Peça p=obterPeçaInformada(); String erro=controlador.inserirPeças(p); if(erro!=null)
erro(erro); else atualizarPeçasCadastradas(p.getCodigo()); }
        catch(IllegalArgumentException e) { erro(e.getMessage()); }
    }
    private void alterar() {
```

```
        try { Peça p=obterPeçaInformada(); String erro=controlador.alterarPeças(p); if(erro!=null)
erro(erro); else atualizarPeçasCadastradas(p.getCodigo()); }
        catch(IllegalArgumentException e) { erro(e.getMessage()); }
    }
    private void consultar() {
        Peça selecionada=(Peça)peçasComboBox.getSelectedItem(); if(selecionada==null)
{ erro("Selecione uma peça.");return; }
        Peça p=Peça.buscarPeças(selecionada.getCodigo()); if(p==null) { erro("Peça não
encontrada.");return; }
        codigoField.setText(Integer.toString(p.getCodigo())); nomeField.setText(p.getNome());
marcaComboBox.setSelectedItem(p.getMarca());
precoField.setText(Double.toString(p.getPreco()));
        sobEncomendaCheckBox.setSelected(p.isSobEncomenda());
        if(p instanceof PeçaMecânica) {
            PeçaMecânica m=(PeçaMecânica)p;
especializaçãoTabbedPane.setSelectedComponent(painelMecânica);
            categoriaMecânicaComboBox.setSelectedItem(m.getCategoriaMecânica());
prazoGarantiaTextField.setText(m.getPrazoGarantia()==null?"":m.getPrazoGarantia().toString());
mecanicoProprioCheckBox.setSelected(m.isMecanicoProprio());
        } else if(p instanceof PeçaAcessório) {
            PeçaAcessório a=(PeçaAcessório)p;
especializaçãoTabbedPane.setSelectedComponent(painelAcessórios);
            categoriaAcessórioComboBox.setSelectedItem(a.getCategoriaAcessório());
corTextField.setText(a.getCor()==null?"":a.getCor());
funileiroProprioCheckBox.setSelected(a.isFunileiroProprio());
        }
    }
    private void remover() {
        Peça p=(Peça)peçasComboBox.getSelectedItem(); if(p==null) { erro("Selecione uma
peça.");return; }
        String msg=controlador.removerPeças(p.getCodigo()); if(msg!=null) erro(msg); else
{ atualizarPeçasCadastradas(-1);limparCampos(); }
    }
    private void limparCampos() {
        codigoField.setText("");nomeField.setText("");marcaComboBox.setSelectedIndex(-
1);precoField.setText("");
        sobEncomendaCheckBox.setSelected(false);
        categoriaMecânicaComboBox.setSelectedIndex(-1); prazoGarantiaTextField.setText("");
mecanicoProprioCheckBox.setSelected(false);
        categoriaAcessórioComboBox.setSelectedIndex(-1); corTextField.setText("");
funileiroProprioCheckBox.setSelected(false); especializaçãoTabbedPane.setSelectedIndex(0);
```

```
    }
    private void erro(String msg)
{ JOptionPane.showMessageDialog(this,msg,"Erro",JOptionPane.ERROR_MESSAGE); }

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private JComboBox<Peça> peçasComboBox;
    private JTextField codigoField, nomeField, precoField;
    private JComboBox<Peça.MarcaPeça> marcaComboBox;
    private JCheckBox sobEncomendaCheckBox;
    private JPanel painelMecânica, painelAcessórios;
    private JComboBox<PeçaMecânica.CategoriaMecânica> categoriaMecânicaComboBox;
    private JTextField prazoGarantiaTextField, corTextField;
    private JCheckBox mecanicoProprioCheckBox, funileiroProprioCheckBox;
    private JComboBox<PeçaAcessório.CategoriaAcessório> categoriaAcessórioComboBox;
    private JLabel categoriaMecânicaLabel, prazoGarantiaLabel, mecanicoProprioLabel;
    private JLabel categoriaAcessórioLabel, corLabel, funileiroProprioLabel;
    private JTabbedPane especializaçãoTabbedPane;
    private JPanel comandosPanel;
    private JButton inserirButton, consultarButton, alterarButton, removerButton, limparButton;
    private JLabel peçasLabel, codigoLabel, nomeLabel, marcaLabel, precoLabel;
    private JPanel camposComunsPanel, abasAlinhadasPanel;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.interfaces.JanelaCadastroPeçasSinistros

```
package interfaces;

import controles.ControladorCadastroPeçasSinistros;
import entidades.Peça;
import entidades.Sinistro;
import java.awt.Frame;
import javax.swing.DefaultComboBoxModel;
import javax.swing.DefaultListModel;
import javax.swing.JOptionPane;

public class JanelaCadastroPeçasSinistros extends javax.swing.JFrame {

   private final ControladorCadastroPeçasSinistros controlador;
   private Sinistro sinistro;
   private Peça[] pecasCadastradas;
   private final DefaultListModel pecasSinistroModel = new DefaultListModel();

   public JanelaCadastroPeçasSinistros(
         ControladorCadastroPeçasSinistros controlador,
         Sinistro sinistro,
         Frame owner
   ) {
      this.controlador = controlador;
      this.sinistro = sinistro;
      pecasCadastradas = Peça.getVisoes();
      initComponents();
      if (owner instanceof JanelaCadastroSinistros) {
         final JanelaCadastroSinistros janelaMae = (JanelaCadastroSinistros) owner;
         addWindowListener(new java.awt.event.WindowAdapter() {
            @Override
            public void windowClosed(java.awt.event.WindowEvent evt) {
               if (sinistro != null) {
                  janelaMae.atualizarListaPeçasSinistro(sinistro.getId());
               }
            }
```

```
        });
    }
    carregarSinistro();
    atualizarPeçasCadastradas();
    atualizarPeçasAssociadas();
    setSize(new java.awt.Dimension(760, 520));
    configurarJanelaDependente(owner);
    limparCampos(null);
}

private void configurarJanelaDependente(Frame owner) {
    setAutoRequestFocus(true);
    setAlwaysOnTop(true);

    if (owner == null) {
        setLocationByPlatform(true);
        return;
    }

    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private void carregarSinistro() {
    if (sinistro == null) {
        sinistroTextField.setText("Sinistro nao informado");
        return;
    }

    sinistroTextField.setText(sinistro.toString());
}

private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(
            this,
            mensagem,
```

```
            "Erro",
            JOptionPane.ERROR_MESSAGE
        );
    }

    private void atualizarPeçasCadastradas() {
        pecasCadastradas = Peça.getVisoes();
        pecasCadastradasComboBox.setModel(
            new DefaultComboBoxModel(pecasCadastradas)
        );
    }

    private void atualizarPeçasAssociadas() {
        pecasSinistroModel.clear();

        if (sinistro == null) {
            pecasSinistroList.setModel(pecasSinistroModel);
            return;
        }

        Peça[] pecas = Peça.buscarPeçasPorSinistro(sinistro.getId());
        sinistro.setPeças(pecas);

        for (Peça peca : pecas) {
            pecasSinistroModel.addElement(peca);
        }

        pecasSinistroList.setModel(pecasSinistroModel);
    }

    private Peça obterPeçaSelecionada() {
        return (Peça) pecasCadastradasComboBox.getSelectedItem();
    }

    private Peça obterPeçaAssociadaSelecionada() {
        return (Peça) pecasSinistroList.getSelectedValue();
    }
```

```
    private void selecionarPeçaNaLista(Peça peca) {
        if (peca == null) return;
        pecasSinistroList.setSelectedValue(peca, true);
    }

    private Peça localizarPeça(int codigo) {
        if (pecasCadastradas == null) return null;
        for (Peça visao : pecasCadastradas) {
            if (visao.getCodigo() == codigo) return visao;
        }
        return null;
    }

    private void limparCampos(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_limparCampos
        if (pecasCadastradasComboBox.getItemCount() > 0) {
            pecasCadastradasComboBox.setSelectedIndex(0);
        } else {
            pecasCadastradasComboBox.setSelectedItem(null);
        }
        pecasSinistroList.clearSelection();
    }//GEN-LAST:event_limparCampos

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {
        java.awt.GridBagConstraints gridBagConstraints;

        comandosPanel = new javax.swing.JPanel();
        inserirPeça = new javax.swing.JButton();
        removerPeça = new javax.swing.JButton();
        limparCampos = new javax.swing.JButton();
        sinistroLabel = new javax.swing.JLabel();
        sinistroTextField = new javax.swing.JTextField();
        pecasCadastradasLabel = new javax.swing.JLabel();
        pecasCadastradasComboBox = new javax.swing.JComboBox();
```

```
pecasSinistroLabel = new javax.swing.JLabel();
pecasSinistroScrollPane = new javax.swing.JScrollPane();
pecasSinistroList = new javax.swing.JList();

setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastro de Peças por Sinistro");
setMinimumSize(new java.awt.Dimension(680, 430));
setPreferredSize(new java.awt.Dimension(760, 520));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirPeça.setText("Inserir");
inserirPeça.addActionListener(new java.awt.event.ActionListener() {
   public void actionPerformed(java.awt.event.ActionEvent evt) {
      inserirPeça(evt);
   }
});
comandosPanel.add(inserirPeça);

removerPeça.setText("Remover");
removerPeça.addActionListener(new java.awt.event.ActionListener() {
   public void actionPerformed(java.awt.event.ActionEvent evt) {
      removerPeça(evt);
   }
});
comandosPanel.add(removerPeça);

limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
   public void actionPerformed(java.awt.event.ActionEvent evt) {
      limparCampos(evt);
   }
});
comandosPanel.add(limparCampos);

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 4;
```

```
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);

sinistroLabel.setText("Sinistro");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(sinistroLabel, gridBagConstraints);

sinistroTextField.setEditable(false);
sinistroTextField.setColumns(28);
sinistroTextField.setPreferredSize(new java.awt.Dimension(280, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(sinistroTextField, gridBagConstraints);

pecasCadastradasLabel.setText("Peças Cadastradas");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasCadastradasLabel, gridBagConstraints);

pecasCadastradasComboBox.setModel(new DefaultComboBoxModel(pecasCadastradas));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(pecasCadastradasComboBox, gridBagConstraints);
```

```
    pecasSinistroLabel.setText("Peças do Sinistro");
    gridBagConstraints = new java.awt.GridBagConstraints();
    gridBagConstraints.gridx = 0;
    gridBagConstraints.gridy = 2;
    gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHEAST;
    gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
    getContentPane().add(pecasSinistroLabel, gridBagConstraints);

    pecasSinistroList.setModel(pecasSinistroModel);
    pecasSinistroList.setSelectionMode(javax.swing.ListSelectionModel.SINGLE_SELECTION);
    pecasSinistroScrollPane.setViewportView(pecasSinistroList);

    gridBagConstraints = new java.awt.GridBagConstraints();
    gridBagConstraints.gridx = 1;
    gridBagConstraints.gridy = 2;
    gridBagConstraints.fill = java.awt.GridBagConstraints.BOTH;
    gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
    gridBagConstraints.weightx = 1.0;
    gridBagConstraints.weighty = 1.0;
    gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
    getContentPane().add(pecasSinistroScrollPane, gridBagConstraints);

    pack();
}// </editor-fold>//GEN-END:initComponents

private void inserirPeça(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_inserirPeça
    Peça peca = obterPeçaSelecionada();
    String mensagemErro = null;

    if (sinistro == null) {
        mensagemErro = "Sinistro nao informado";
    } else if (peca == null) {
        mensagemErro = "Nenhuma Peça selecionada";
    } else {
        mensagemErro = controlador.inserirPeçasSinistros(peca, sinistro);
    }
```

```
    if (mensagemErro == null) {
        atualizarPeçasAssociadas();
        selecionarPeçaNaLista(localizarPeça(peca.getCodigo()));
    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_inserirPeça

private void removerPeça(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_removerPeça
    Peça peca = obterPeçaAssociadaSelecionada();
    String mensagemErro = null;

    if (sinistro == null) {
        mensagemErro = "Sinistro nao informado";
    } else if (peca == null) {
        mensagemErro = "Nenhuma Peça selecionada";
    } else {
        mensagemErro = controlador.removerPeçasSinistros(peca, sinistro);
    }

    if (mensagemErro == null) {
        atualizarPeçasAssociadas();
        pecasSinistroList.clearSelection();
    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_removerPeça

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton inserirPeça;
private javax.swing.JButton limparCampos;
private javax.swing.JComboBox pecasCadastradasComboBox;
private javax.swing.JLabel pecasCadastradasLabel;
private javax.swing.JLabel pecasSinistroLabel;
private javax.swing.JList pecasSinistroList;
private javax.swing.JScrollPane pecasSinistroScrollPane;
```

```
    private javax.swing.JButton removerPeça;
    private javax.swing.JLabel sinistroLabel;
    private javax.swing.JTextField sinistroTextField;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.interfaces.JanelaCadastroSeguradoras

```
package interfaces;

import javax.swing.JOptionPane;
import javax.swing.ButtonGroup;
import controles.ControladorCadastroSeguradoras;
import entidades.Seguradora;
import javax.swing.DefaultComboBoxModel;
import java.awt.Frame;

public class JanelaCadastroSeguradoras extends javax.swing.JFrame {

   ControladorCadastroSeguradoras controlador;
   Seguradora[] seguradorasCadastradas;

   public JanelaCadastroSeguradoras(ControladorCadastroSeguradoras controlador) {
      this(controlador, null);
   }

   public JanelaCadastroSeguradoras(
         ControladorCadastroSeguradoras controlador,
         Frame owner
   ) {
      this.controlador = controlador;
      seguradorasCadastradas = Seguradora.getVisoes();
      initComponents();
      atualizarSeguradorasCadastradas();
      setSize(new java.awt.Dimension(760, 540));
      configurarJanelaDependente(owner);
      limparCampos(null);
   }

   private void configurarJanelaDependente(Frame owner) {
      setAutoRequestFocus(true);
      setAlwaysOnTop(true);
```

```
    if (owner == null) {
        setLocationByPlatform(true);
        return;
    }

    int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
    int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
    setLocation(x, y);
}

private Seguradora localizarSeguradora(String nome) {
    if (nome == null) return null;
    for (Seguradora visao : seguradorasCadastradas) {
        if (nome.equals(visao.getNome())) return visao;
    }
    return null;
}

private void atualizarSeguradorasCadastradas() {
    atualizarSeguradorasCadastradas(null);
}

private void atualizarSeguradorasCadastradas(String nomeSelecionado) {
    seguradorasCadastradas = Seguradora.getVisoes();
    seguradorasCadastradasComboBox.setModel(
            new DefaultComboBoxModel(seguradorasCadastradas)
    );

    Seguradora visaoSelecionada = localizarSeguradora(nomeSelecionado);
    if (visaoSelecionada != null) {
        seguradorasCadastradasComboBox.setSelectedItem(visaoSelecionada);
    }
}

private void informarErro(String mensagem) {
    JOptionPane.showMessageDialog(
        this,
```

```
            mensagem,
            "Erro",
            JOptionPane.ERROR_MESSAGE
        );
    }

    private Boolean obterPossuiAtendimento24hInformado() {
        if (possuiAtendimento24hSimRadioButton.isSelected()) return true;
        if (possuiAtendimento24hNaoRadioButton.isSelected()) return false;
        return null;
    }

    private void definirPossuiAtendimento24h(Boolean valor) {
        possuiAtendimento24hButtonGroup.clearSelection();
        if (valor == null) return;

        if (valor) {
            possuiAtendimento24hSimRadioButton.setSelected(true);
        } else {
            possuiAtendimento24hNaoRadioButton.setSelected(true);
        }
    }

    private Seguradora.FormaPagamentoPreferencial
obterFormaPagamentoPreferencialSelecionada() {
        Object selecionado = formaPagamentoPreferencialComboBox.getSelectedItem();
        if (selecionado == null) {
            return null;
        }

        return Seguradora.FormaPagamentoPreferencial.fromTexto(selecionado.toString());
    }

    private Seguradora obterSeguradoraInformada() {

        String nome = nomeTextField.getText();
        if (nome.isEmpty()) return null;
```

```
    String cidade = cidadeTextField.getText();
    if (cidade.isEmpty()) cidade = null;

    String coberturaStr = coberturaPercentualTextField.getText();
    if (coberturaStr.isEmpty()) return null;

    double coberturaPercentual;

    try {
        coberturaPercentual =
                Double.parseDouble(coberturaStr.replace(",", "."));
    } catch (NumberFormatException e) {
        return null;
    }

    Boolean possuiAtendimento24h =
            obterPossuiAtendimento24hInformado();
    if (possuiAtendimento24h == null) return null;

    Seguradora.FormaPagamentoPreferencial formaPagamentoPreferencial =
            obterFormaPagamentoPreferencialSelecionada();
    if (formaPagamentoPreferencial == null) return null;

    return new Seguradora(
        nome,
        cidade,
        coberturaPercentual,
        possuiAtendimento24h,
        formaPagamentoPreferencial
    );
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;
```

```
        comandosPanel = new javax.swing.JPanel();
        inserirSeguradora = new javax.swing.JButton();
        alterarSeguradora = new javax.swing.JButton();
        consultarSeguradora = new javax.swing.JButton();
        removerSeguradora = new javax.swing.JButton();
        limparCampos = new javax.swing.JButton();
        nomeLabel = new javax.swing.JLabel();
        nomeTextField = new javax.swing.JTextField();
        cidadeLabel = new javax.swing.JLabel();
        cidadeTextField = new javax.swing.JTextField();
        coberturaPercentualLabel = new javax.swing.JLabel();
        coberturaPercentualTextField = new javax.swing.JTextField();
        possuiAtendimento24hLabel = new javax.swing.JLabel();
        possuiAtendimento24hPanel = new javax.swing.JPanel();
        possuiAtendimento24hSimRadioButton = new javax.swing.JRadioButton();
        possuiAtendimento24hNaoRadioButton = new javax.swing.JRadioButton();
        possuiAtendimento24hButtonGroup = new javax.swing.ButtonGroup();
        formaPagamentoPreferencialComboBox = new javax.swing.JComboBox();
        formaPagamentoPreferencialComboBox.setModel(new
DefaultComboBoxModel(Seguradora.FormaPagamentoPreferencial.values()));
        formaPagamentoPreferencialLabel = new javax.swing.JLabel();
        seguradorasCadastradasLabel = new javax.swing.JLabel();
        seguradorasCadastradasComboBox = new javax.swing.JComboBox();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Cadastrar Seguradoras");
        setMinimumSize(new java.awt.Dimension(620, 360));
        setPreferredSize(new java.awt.Dimension(700, 460));
        getContentPane().setLayout(new java.awt.GridBagLayout());

        inserirSeguradora.setText("Inserir");
        inserirSeguradora.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                inserirSeguradora(evt);
            }
        });
```

```
comandosPanel.add(inserirSeguradora);

alterarSeguradora.setText("Alterar");
alterarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSeguradora(evt);
    }
});
comandosPanel.add(alterarSeguradora);

consultarSeguradora.setText("Consultar");
consultarSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSeguradora(evt);
    }
});
comandosPanel.add(consultarSeguradora);

removerSeguradora.setText("Remover");
removerSeguradora.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerSeguradora(evt);
    }
});
comandosPanel.add(removerSeguradora);

limparCampos.setText("Limpar");
limparCampos.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        limparCampos(evt);
    }
});
comandosPanel.add(limparCampos);

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 6;
```

```
gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
getContentPane().add(comandosPanel, gridBagConstraints);

nomeLabel.setText("Nome");
nomeLabel.setMaximumSize(null);
nomeLabel.setMinimumSize(null);
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(nomeLabel, gridBagConstraints);

nomeTextField.setColumns(24);
nomeTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.ipadx = 80;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(nomeTextField, gridBagConstraints);

cidadeLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
getContentPane().add(cidadeLabel, gridBagConstraints);

cidadeTextField.setColumns(18);
cidadeTextField.setPreferredSize(new java.awt.Dimension(180, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.ipadx = 20;
```

```
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(cidadeTextField, gridBagConstraints);

        coberturaPercentualLabel.setText("Cobertura Percentual");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 3;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
        getContentPane().add(coberturaPercentualLabel, gridBagConstraints);

        coberturaPercentualTextField.setColumns(10);
        coberturaPercentualTextField.setPreferredSize(new java.awt.Dimension(100, 20));
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 3;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(coberturaPercentualTextField, gridBagConstraints);

        possuiAtendimento24hLabel.setText("Possui Atendimento 24h");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 4;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
        getContentPane().add(possuiAtendimento24hLabel, gridBagConstraints);

        possuiAtendimento24hPanel.setLayout(new
java.awt.FlowLayout(java.awt.FlowLayout.LEFT));

        possuiAtendimento24hButtonGroup.add(possuiAtendimento24hSimRadioButton);
        possuiAtendimento24hButtonGroup.add(possuiAtendimento24hNaoRadioButton);

        possuiAtendimento24hSimRadioButton.setText("Sim");
        possuiAtendimento24hPanel.add(possuiAtendimento24hSimRadioButton);
```

```
        possuiAtendimento24hNaoRadioButton.setText("Não");
        possuiAtendimento24hPanel.add(possuiAtendimento24hNaoRadioButton);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 4;
        gridBagConstraints.gridwidth = 2;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(possuiAtendimento24hPanel, gridBagConstraints);

        formaPagamentoPreferencialLabel.setText("Forma de Pagamento Preferencial");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 5;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(1, 16, 10, 10);
        getContentPane().add(formaPagamentoPreferencialLabel, gridBagConstraints);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 5;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(formaPagamentoPreferencialComboBox, gridBagConstraints);

        seguradorasCadastradasLabel.setText("Seguradoras Cadastradas");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(0, 12, 0, 5);
        getContentPane().add(seguradorasCadastradasLabel, gridBagConstraints);

        seguradorasCadastradasComboBox.setModel(new
DefaultComboBoxModel(seguradorasCadastradas));
        seguradorasCadastradasComboBox.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
```

```
            seguradorasCadastradasComboBoxActionPerformed(evt);
        }
    });
    gridBagConstraints = new java.awt.GridBagConstraints();
    gridBagConstraints.gridx = 1;
    gridBagConstraints.gridy = 0;
    gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
    gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
    getContentPane().add(seguradorasCadastradasComboBox, gridBagConstraints);

    pack();
}// </editor-fold>//GEN-END:initComponents

private void alterarSeguradora(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_alterarSeguradora
    Seguradora seguradora =
            obterSeguradoraInformada();

    String mensagemErro = null;

    if (seguradora != null)
        mensagemErro =
                controlador.alterarSeguradora(seguradora);
    else
        mensagemErro =
                "Algum atributo da Seguradora não foi informado.";

    if (mensagemErro == null) {

        Seguradora visao =
                (Seguradora) seguradorasCadastradasComboBox.getSelectedItem();

        if (visao != null) {

            visao.setCidade(
                    seguradora.getCidade()
            );
```

```
            seguradorasCadastradasComboBox.updateUI();
        }

    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_alterarSeguradora

private void inserirSeguradora(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_inserirSeguradora
    Seguradora seguradora =
            obterSeguradoraInformada();

    String mensagemErro = null;

    if (seguradora != null)
        mensagemErro =
                controlador.inserirSeguradora(seguradora);
    else
        mensagemErro =
                "Algum atributo da Seguradora não foi informado";

    if (mensagemErro == null) {

        Seguradora visao = seguradora.getVisao();
        atualizarSeguradorasCadastradas(visao.getNome());

    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_inserirSeguradora

private void consultarSeguradora(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_consultarSeguradora
    Seguradora visao =
            (Seguradora)
```

```
        seguradorasCadastradasComboBox
                .getSelectedItem();

Seguradora seguradora = null;

String mensagemErro = null;

if (visao != null) {

    seguradora =
            Seguradora.buscarSeguradora(
                    visao.getNome()
            );

    if (seguradora == null)
        mensagemErro =
                "Seguradora não cadastrada";

} else {
    mensagemErro =
            "Nenhuma Seguradora selecionada";
}

if (mensagemErro == null) {

    nomeTextField.setText(
            seguradora.getNome()
    );

    String cidade =
            seguradora.getCidade();

    if (cidade == null)
        cidade = "";

    cidadeTextField.setText(cidade);
```

```
            coberturaPercentualTextField.setText(
                    String.valueOf(
                            seguradora
                                    .getCoberturaPercentual()
                    )
            );

            definirPossuiAtendimento24h(seguradora.getPossuiAtendimento24h());

            formaPagamentoPreferencialComboBox.setSelectedItem(
                    seguradora.getFormaPagamentoPreferencial().toString()
            );

        } else {
            informarErro(mensagemErro);
        }
    }//GEN-LAST:event_consultarSeguradora

    private void removerSeguradora(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_removerSeguradora
        Seguradora visao =
                (Seguradora)
                seguradorasCadastradasComboBox
                        .getSelectedItem();

        String mensagemErro = null;

        if (visao != null)
            mensagemErro =
                    controlador.removerSeguradora(
                            visao.getNome()
                    );
        else
            mensagemErro =
                    "Nenhuma Seguradora selecionada";

        if (mensagemErro == null) {
```

```
            atualizarSeguradorasCadastradas();

            limparCampos(evt);

        } else {
            informarErro(mensagemErro);
        }
    }//GEN-LAST:event_removerSeguradora

    private void limparCampos(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_limparCampos
        nomeTextField.setText("");
        cidadeTextField.setText("");
        coberturaPercentualTextField.setText("");
        possuiAtendimento24hButtonGroup.clearSelection();
        formaPagamentoPreferencialComboBox.setSelectedIndex(0);
    }//GEN-LAST:event_limparCampos

    private void seguradorasCadastradasComboBoxActionPerformed(java.awt.event.ActionEvent
evt) {//GEN-FIRST:event_seguradoras_cadastradasComboBoxActionPerformed

    }//GEN-LAST:event_seguradoras_cadastradasComboBoxActionPerformed



    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JButton alterarSeguradora;
    private javax.swing.JLabel cidadeLabel;
    private javax.swing.JTextField cidadeTextField;
    private javax.swing.JLabel coberturaPercentualLabel;
    private javax.swing.JTextField coberturaPercentualTextField;
    private javax.swing.JPanel comandosPanel;
    private javax.swing.JButton consultarSeguradora;
    private javax.swing.JButton inserirSeguradora;
    private javax.swing.JButton limparCampos;
    private javax.swing.JLabel nomeLabel;
    private javax.swing.JTextField nomeTextField;
```

```
    private javax.swing.JLabel possuiAtendimento24hLabel;
    private javax.swing.ButtonGroup possuiAtendimento24hButtonGroup;
    private javax.swing.JRadioButton possuiAtendimento24hNaoRadioButton;
    private javax.swing.JPanel possuiAtendimento24hPanel;
    private javax.swing.JRadioButton possuiAtendimento24hSimRadioButton;
    private javax.swing.JComboBox formaPagamentoPreferencialComboBox;
    private javax.swing.JLabel formaPagamentoPreferencialLabel;
    private javax.swing.JButton removerSeguradora;
    private javax.swing.JComboBox seguradorasCadastradasComboBox;
    private javax.swing.JLabel seguradorasCadastradasLabel;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.interfaces.JanelaCadastroSinistros

```
package interfaces;

import controles.ControladorCadastroPeçasSinistros;
import controles.ControladorCadastroSinistros;
import entidades.Sinistro;
import java.awt.Frame;
import javax.swing.DefaultComboBoxModel;
import javax.swing.DefaultListModel;
import javax.swing.JOptionPane;

public class JanelaCadastroSinistros extends javax.swing.JFrame {

    ControladorCadastroSinistros controlador;
    Sinistro[] sinistrosCadastrados;
    private final DefaultListModel pecasSinistroModel = new DefaultListModel();

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador) {
        this(controlador, null);
    }

    public JanelaCadastroSinistros(ControladorCadastroSinistros controlador, Frame owner) {
        this.controlador = controlador;
        sinistrosCadastrados = Sinistro.getVisoes();
        initComponents();
        atualizarSinistrosCadastrados();
        setSize(new java.awt.Dimension(820, 620));
        configurarJanelaDependente(owner);
        limparCampos(null);
    }

    private void configurarJanelaDependente(Frame owner) {
        setAutoRequestFocus(true);
        setAlwaysOnTop(true);

        if (owner == null) {
```

```
            setLocationByPlatform(true);
            return;
        }

        int x = owner.getX() + (owner.getWidth() - getWidth()) / 2;
        int y = owner.getY() + (owner.getHeight() - getHeight()) / 2;
        setLocation(x, y);
    }

    private Sinistro localizarSinistro(int id) {
        if (sinistrosCadastrados == null) return null;
        for (Sinistro visao : sinistrosCadastrados) {
            if (visao.getId() == id) return visao;
        }
        return null;
    }

    private void atualizarSinistrosCadastrados() {
        atualizarSinistrosCadastrados(0);
    }

    private void atualizarSinistrosCadastrados(int idSelecionado) {
        sinistrosCadastrados = Sinistro.getVisoes();
        sinistrosCadastradosComboBox.setModel(new
DefaultComboBoxModel(sinistrosCadastrados));

        Sinistro visaoSelecionada = localizarSinistro(idSelecionado);
        if (visaoSelecionada != null) {
            sinistrosCadastradosComboBox.setSelectedItem(visaoSelecionada);
        }
    }

    private void informarErro(String mensagem) {
        JOptionPane.showMessageDialog(this, mensagem, "Erro",
JOptionPane.ERROR_MESSAGE);
    }
```

```
public void atualizarListaPeçasSinistro(int sinistroId) {
    pecasSinistroModel.clear();
    if (sinistroId <= 0) {
        pecasSinistroList.setModel(pecasSinistroModel);
        return;
    }

    for (entidades.Peça peca : entidades.Peça.buscarPeçasPorSinistro(sinistroId)) {
        pecasSinistroModel.addElement(peca);
    }
    pecasSinistroList.setModel(pecasSinistroModel);
}

private Boolean obterPerdaTotalInformada() {
    return perdaTotalCheckBox.isSelected();
}

private void definirPerdaTotal(Boolean valor) {
    perdaTotalCheckBox.setSelected(Boolean.TRUE.equals(valor));
}

private Sinistro obterSinistroInformado() {
    String segurado = seguradoTextField.getText().trim();
    if (segurado.isEmpty()) return null;

    String telefone = telefoneTextField.getText().trim();
    if (telefone.isEmpty()) return null;

    String cidade = clienteTextField.getText().trim();
    if (cidade.isEmpty()) cidade = null;

    Sinistro.GrauMonta grauMonta =
            (Sinistro.GrauMonta) grauMontaComboBox.getSelectedItem();
    if (grauMonta == null) return null;

    Boolean perdaTotal = obterPerdaTotalInformada();
    if (perdaTotal == null) return null;
```

```
    int id = numeroTextField.getText().trim().isEmpty() ? 0
            : Integer.parseInt(numeroTextField.getText().trim());
    return new Sinistro(id, segurado, telefone, cidade, grauMonta, perdaTotal);
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    comandosPanel = new javax.swing.JPanel();
    inserirSinistro = new javax.swing.JButton();
    alterarSinistro = new javax.swing.JButton();
    consultarSinistro = new javax.swing.JButton();
    removerSinistro = new javax.swing.JButton();
    pecasSinistro = new javax.swing.JButton();
    limparCampos = new javax.swing.JButton();
    sinistrosCadastradosLabel = new javax.swing.JLabel();
    sinistrosCadastradosComboBox = new javax.swing.JComboBox();
    numeroLabel = new javax.swing.JLabel();
    numeroTextField = new javax.swing.JTextField();
    seguradoLabel = new javax.swing.JLabel();
    seguradoTextField = new javax.swing.JTextField();
    clienteLabel = new javax.swing.JLabel();
    clienteTextField = new javax.swing.JTextField();
    telefoneLabel = new javax.swing.JLabel();
    telefoneTextField = new javax.swing.JTextField();
    grauMontaLabel = new javax.swing.JLabel();
    grauMontaComboBox = new javax.swing.JComboBox();
    perdaTotalLabel = new javax.swing.JLabel();
    perdaTotalPanel = new javax.swing.JPanel();
    perdaTotalCheckBox = new javax.swing.JCheckBox();
    pecasSinistroLabel = new javax.swing.JLabel();
    pecasSinistroScrollPane = new javax.swing.JScrollPane();
    pecasSinistroList = new javax.swing.JList();
```

```
setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
setTitle("Cadastrar Sinistros");
setMinimumSize(new java.awt.Dimension(680, 420));
setPreferredSize(new java.awt.Dimension(820, 620));
getContentPane().setLayout(new java.awt.GridBagLayout());

inserirSinistro.setText("Inserir");
inserirSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        inserirSinistro(evt);
    }
});
comandosPanel.add(inserirSinistro);

alterarSinistro.setText("Alterar");
alterarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        alterarSinistro(evt);
    }
});
comandosPanel.add(alterarSinistro);

consultarSinistro.setText("Consultar");
consultarSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        consultarSinistro(evt);
    }
});
comandosPanel.add(consultarSinistro);

removerSinistro.setText("Remover");
removerSinistro.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        removerSinistro(evt);
    }
});
comandosPanel.add(removerSinistro);
```

```
        pecasSinistro.setText("Peças");
        pecasSinistro.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                pecasSinistro(evt);
            }
        });
        comandosPanel.add(pecasSinistro);

        limparCampos.setText("Limpar");
        limparCampos.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                limparCampos(evt);
            }
        });
        comandosPanel.add(limparCampos);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 8;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHWEST;
        gridBagConstraints.insets = new java.awt.Insets(5, 5, 5, 5);
        getContentPane().add(comandosPanel, gridBagConstraints);

        sinistrosCadastradosLabel.setText("Sinistros Cadastrados");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(sinistrosCadastradosLabel, gridBagConstraints);

        sinistrosCadastradosComboBox.setModel(new
DefaultComboBoxModel(sinistrosCadastrados));
        sinistrosCadastradosComboBox.addActionListener(new java.awt.event.ActionListener() {
            public void actionPerformed(java.awt.event.ActionEvent evt) {
                sinistrosCadastradosComboBoxActionPerformed(evt);
```

```
    }
});
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(sinistrosCadastradosComboBox, gridBagConstraints);

numeroLabel.setText("ID");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(numeroLabel, gridBagConstraints);

numeroTextField.setEditable(false);
numeroTextField.setColumns(8);
numeroTextField.setMinimumSize(new java.awt.Dimension(80, 20));
numeroTextField.setPreferredSize(new java.awt.Dimension(80, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 1;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(numeroTextField, gridBagConstraints);

seguradoLabel.setText("Segurado");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoLabel, gridBagConstraints);

seguradoTextField.setColumns(24);
```

```
seguradoTextField.setMinimumSize(new java.awt.Dimension(240, 20));
seguradoTextField.setPreferredSize(new java.awt.Dimension(240, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(seguradoTextField, gridBagConstraints);

clienteLabel.setText("Cidade");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(clienteLabel, gridBagConstraints);

clienteTextField.setColumns(16);
clienteTextField.setMinimumSize(new java.awt.Dimension(160, 20));
clienteTextField.setPreferredSize(new java.awt.Dimension(160, 20));
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 3;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(clienteTextField, gridBagConstraints);

telefoneLabel.setText("Telefone");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 4;
gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
getContentPane().add(telefoneLabel, gridBagConstraints);

telefoneTextField.setColumns(12);
telefoneTextField.setMinimumSize(new java.awt.Dimension(120, 20));
```

```
        telefoneTextField.setPreferredSize(new java.awt.Dimension(120, 20));
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 4;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(telefoneTextField, gridBagConstraints);

        grauMontaLabel.setText("Grau de Monta");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 5;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(grauMontaLabel, gridBagConstraints);

        grauMontaComboBox.setModel(new
DefaultComboBoxModel(Sinistro.GrauMonta.values()));
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 5;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(grauMontaComboBox, gridBagConstraints);

        perdaTotalLabel.setText("Perda Total");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 6;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.EAST;
        gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
        getContentPane().add(perdaTotalLabel, gridBagConstraints);

        perdaTotalPanel.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));

        perdaTotalCheckBox.setText("Sim");
        perdaTotalPanel.add(perdaTotalCheckBox);
```

```
    gridBagConstraints = new java.awt.GridBagConstraints();
    gridBagConstraints.gridx = 1;
    gridBagConstraints.gridy = 6;
    gridBagConstraints.gridwidth = 2;
    gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
    gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
    getContentPane().add(perdaTotalPanel, gridBagConstraints);

    pecasSinistroLabel.setText("Peças do Sinistro");
    gridBagConstraints = new java.awt.GridBagConstraints();
    gridBagConstraints.gridx = 0;
    gridBagConstraints.gridy = 7;
    gridBagConstraints.anchor = java.awt.GridBagConstraints.NORTHEAST;
    gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
    getContentPane().add(pecasSinistroLabel, gridBagConstraints);

    pecasSinistroScrollPane.setPreferredSize(new java.awt.Dimension(300, 90));

    pecasSinistroList.setSelectionMode(javax.swing.ListSelectionModel.SINGLE_SELECTION);
    pecasSinistroList.setModel(pecasSinistroModel);
    pecasSinistroScrollPane.setViewportView(pecasSinistroList);

    gridBagConstraints = new java.awt.GridBagConstraints();
    gridBagConstraints.gridx = 1;
    gridBagConstraints.gridy = 7;
    gridBagConstraints.fill = java.awt.GridBagConstraints.BOTH;
    gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
    gridBagConstraints.weightx = 1.0;
    gridBagConstraints.weighty = 1.0;
    gridBagConstraints.insets = new java.awt.Insets(1, 5, 10, 5);
    getContentPane().add(pecasSinistroScrollPane, gridBagConstraints);

    pack();
}// </editor-fold>//GEN-END:initComponents

private void sinistrosCadastradosComboBoxActionPerformed(java.awt.event.ActionEvent evt)
```

```
{//GEN-FIRST:event_sinistros_cadastradosComboBoxActionPerformed
        Sinistro visao = (Sinistro) sinistrosCadastradosComboBox.getSelectedItem();
        atualizarListaPeçasSinistro(visao != null ? visao.getId() : 0);
    }//GEN-LAST:event_sinistros_cadastradosComboBoxActionPerformed


    private void limparCampos(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_limparCampos
        numeroTextField.setText("");
        seguradoTextField.setText("");
        clienteTextField.setText("");
        telefoneTextField.setText("");
        grauMontaComboBox.setSelectedIndex(-1);
        perdaTotalCheckBox.setSelected(false);
        atualizarListaPeçasSinistro(0);
    }//GEN-LAST:event_limparCampos


    private void consultarSinistro(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_consultarSinistro
        Sinistro visao = (Sinistro) sinistrosCadastradosComboBox.getSelectedItem();
        String mensagemErro = null;
        if (visao == null) {
            mensagemErro = "Nenhum Sinistro selecionado";
        } else {
            Sinistro sinistro = Sinistro.buscarSinistro(visao.getId());
            if (sinistro == null) {
                mensagemErro = "Sinistro não cadastrado";
            } else {
                numeroTextField.setText(String.valueOf(sinistro.getId()));
                seguradoTextField.setText(sinistro.getSegurado());
                clienteTextField.setText(sinistro.getCidade());
                telefoneTextField.setText(sinistro.getTelefone());
                grauMontaComboBox.setSelectedItem(sinistro.getGrauMonta());
                definirPerdaTotal(sinistro.getPerdaTotal());
                atualizarListaPeçasSinistro(sinistro.getId());
            }
        }
```

```
    if (mensagemErro != null) informarErro(mensagemErro);
}//GEN-LAST:event_consultarSinistro

private void inserirSinistro(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_inserirSinistro
    Sinistro sinistro = obterSinistroInformado();
    String mensagemErro = null;

    if (sinistro != null) {
        mensagemErro = controlador.inserirSinistro(sinistro);
    } else {
        mensagemErro = "Algum atributo do Sinistro não foi informado";
    }

    if (mensagemErro == null) {
        atualizarSinistrosCadastrados(sinistro.getId());
        atualizarListaPeçasSinistro(sinistro.getId());
        numeroTextField.setText(String.valueOf(sinistro.getId()));
    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_inserirSinistro

private void alterarSinistro(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_alterarSinistro
    Sinistro sinistro = obterSinistroInformado();
    String mensagemErro = null;

    if (sinistro != null) {
        mensagemErro = controlador.alterarSinistro(sinistro);
    } else {
        mensagemErro = "Algum atributo do Sinistro não foi informado";
    }

    if (mensagemErro == null) {
        Sinistro visao = (Sinistro) sinistrosCadastradosComboBox.getSelectedItem();

        if (visao != null) {
            visao.setSegurado(sinistro.getSegurado());
```

```
            visao.setTelefone(sinistro.getTelefone());
            visao.setCidade(sinistro.getCidade());
            visao.setGrauMonta(sinistro.getGrauMonta());
            visao.setPerdaTotal(sinistro.getPerdaTotal());

            sinistrosCadastradosComboBox.updateUI();
        }
    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_alterarSinistro

private void removerSinistro(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_removerSinistro
    Sinistro visao = (Sinistro) sinistrosCadastradosComboBox.getSelectedItem();
    String mensagemErro = null;

    if (visao != null) {
        mensagemErro = controlador.removerSinistro(visao.getId());
    } else {
        mensagemErro = "Nenhum Sinistro selecionado";
    }

    if (mensagemErro == null) {
        atualizarSinistrosCadastrados();
        limparCampos(evt);
    } else {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_removerSinistro

private void pecasSinistro(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_pecasSinistro
    Sinistro visao = (Sinistro) sinistrosCadastradosComboBox.getSelectedItem();
    String mensagemErro = null;

    if (visao == null) {
        mensagemErro = "Nenhum Sinistro selecionado";
```

```
    } else {
        Sinistro sinistro = Sinistro.buscarSinistro(visao.getId());
        if (sinistro == null) {
            mensagemErro = "Sinistro não cadastrado";
        } else {
            new ControladorCadastroPeçasSinistros(this, sinistro);
            atualizarListaPeçasSinistro(sinistro.getId());
        }
    }

    if (mensagemErro != null) {
        informarErro(mensagemErro);
    }
}//GEN-LAST:event_pecasSinistro

// Variables declaration - do not modify//GEN-BEGIN:variables
private javax.swing.JButton alterarSinistro;
private javax.swing.JLabel clienteLabel;
private javax.swing.JTextField clienteTextField;
private javax.swing.JPanel comandosPanel;
private javax.swing.JButton consultarSinistro;
private javax.swing.JComboBox grauMontaComboBox;
private javax.swing.JLabel grauMontaLabel;
private javax.swing.JButton inserirSinistro;
private javax.swing.JButton limparCampos;
private javax.swing.JLabel numeroLabel;
private javax.swing.JTextField numeroTextField;
private javax.swing.JButton pecasSinistro;
private javax.swing.JLabel pecasSinistroLabel;
private javax.swing.JList pecasSinistroList;
private javax.swing.JScrollPane pecasSinistroScrollPane;
private javax.swing.JCheckBox perdaTotalCheckBox;
private javax.swing.JLabel perdaTotalLabel;
private javax.swing.JPanel perdaTotalPanel;
private javax.swing.JButton removerSinistro;
private javax.swing.JLabel seguradoLabel;
private javax.swing.JTextField seguradoTextField;
```

```
    private javax.swing.JComboBox sinistrosCadastradosComboBox;
    private javax.swing.JLabel sinistrosCadastradosLabel;
    private javax.swing.JLabel telefoneLabel;
    private javax.swing.JTextField telefoneTextField;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.interfaces.JanelaPesquisaOrçamentos

```
package interfaces;

import entidades.Orçamento;
import entidades.Peça;
import entidades.Sinistro;
import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import java.sql.Timestamp;
import java.util.ArrayList;
import javax.swing.JButton;
import javax.swing.ButtonGroup;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JRadioButton;
import javax.swing.JScrollPane;
import javax.swing.JTabbedPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;

public class JanelaPesquisaOrçamentos extends JFrame {
    public JanelaPesquisaOrçamentos() {
        initComponents();
        configurarEventos();
        limparFiltros();
        setLocationByPlatform(true);
    }

    private void configurarEventos() {
        atendimento24hGrupo.add(atendimento24hSim);
```

```
    atendimento24hGrupo.add(atendimento24hNao);
    pesquisarButton.addActionListener(e -> pesquisar());
    limparButton.addActionListener(e -> limparFiltros());
    limparPesquisaButton.addActionListener(e -> resultados.setText(""));
}

@SuppressWarnings("unchecked")
// <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
private void initComponents() {
    java.awt.GridBagConstraints gridBagConstraints;

    atendimento24hGrupo = new javax.swing.ButtonGroup();
    filtros = new javax.swing.JPanel();
    painelOrçamento = new javax.swing.JPanel();
    dataLabel = new javax.swing.JLabel();
    dataMinimaOrçamento = new javax.swing.JTextField();
    painelSeguradora = new javax.swing.JPanel();
    atendimentoLabel = new javax.swing.JLabel();
    atendimento24h = new javax.swing.JPanel();
    atendimento24hSim = new javax.swing.JRadioButton();
    atendimento24hNao = new javax.swing.JRadioButton();
    painelSinistro = new javax.swing.JPanel();
    grauLabel = new javax.swing.JLabel();
    grauMonta = new javax.swing.JComboBox();
    painelPeças = new javax.swing.JPanel();
    marcaLabel = new javax.swing.JLabel();
    marcaPeça = new javax.swing.JComboBox();
    todosObjetosCheckBox = new javax.swing.JCheckBox();
    subclassesTabbedPane = new javax.swing.JTabbedPane();
    painelMecânica = new javax.swing.JPanel();
    prazoLabel = new javax.swing.JLabel();
    prazoMínimoGarantia = new javax.swing.JTextField();
    painelAcessórios = new javax.swing.JPanel();
    corLabel = new javax.swing.JLabel();
    corAcessório = new javax.swing.JTextField();
    resultadosScrollPane = new javax.swing.JScrollPane();
    resultados = new javax.swing.JTextArea();
```

```
        botoes = new javax.swing.JPanel();
        pesquisarButton = new javax.swing.JButton();
        limparButton = new javax.swing.JButton();
        limparPesquisaButton = new javax.swing.JButton();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Pesquisar Orçamentos");
        setMinimumSize(new java.awt.Dimension(600, 800));
        setPreferredSize(new java.awt.Dimension(600, 810));
        getContentPane().setLayout(new java.awt.GridBagLayout());

        filtros.setLayout(new java.awt.GridBagLayout());

        painelOrçamento.setBorder(javax.swing.BorderFactory.createTitledBorder("Filtro de
Orçamento"));
        painelOrçamento.setLayout(new java.awt.GridBagLayout());

        dataLabel.setText("Data mínima (AAAA-MM-DD HH:MM:SS)");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelOrçamento.add(dataLabel, gridBagConstraints);

        dataMinimaOrçamento.setColumns(16);
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelOrçamento.add(dataMinimaOrçamento, gridBagConstraints);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.fill = java.awt.GridBagConstraints.HORIZONTAL;
        gridBagConstraints.weightx = 1.0;
        filtros.add(painelOrçamento, gridBagConstraints);
```

```
        painelSeguradora.setBorder(javax.swing.BorderFactory.createTitledBorder("Filtro de Seguradora"));
        painelSeguradora.setLayout(new java.awt.GridBagLayout());

        atendimentoLabel.setText("Seguradora possui atendimento 24h");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelSeguradora.add(atendimentoLabel, gridBagConstraints);

        atendimento24h.setLayout(new java.awt.FlowLayout(java.awt.FlowLayout.LEFT));

        atendimento24hSim.setText("Sim");
        atendimento24h.add(atendimento24hSim);

        atendimento24hNao.setText("Não");
        atendimento24h.add(atendimento24hNao);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelSeguradora.add(atendimento24h, gridBagConstraints);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 1;
        gridBagConstraints.fill = java.awt.GridBagConstraints.HORIZONTAL;
        gridBagConstraints.weightx = 1.0;
        filtros.add(painelSeguradora, gridBagConstraints);

        painelSinistro.setBorder(javax.swing.BorderFactory.createTitledBorder("Filtro de Sinistro"));
        painelSinistro.setLayout(new java.awt.GridBagLayout());

        grauLabel.setText("Grau da monta");
```

```
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelSinistro.add(grauLabel, gridBagConstraints);

        grauMonta.setModel(new
javax.swing.DefaultComboBoxModel(Sinistro.GrauMonta.values()));
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelSinistro.add(grauMonta, gridBagConstraints);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 2;
        gridBagConstraints.fill = java.awt.GridBagConstraints.HORIZONTAL;
        gridBagConstraints.weightx = 1.0;
        filtros.add(painelSinistro, gridBagConstraints);

        painelPeças.setBorder(javax.swing.BorderFactory.createTitledBorder("Filtros de Peças"));
        painelPeças.setLayout(new java.awt.GridBagLayout());

        marcaLabel.setText("Marca da peça");
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelPeças.add(marcaLabel, gridBagConstraints);

        marcaPeça.setModel(new javax.swing.DefaultComboBoxModel(Peça.MarcaPeça.values()));
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 1;
        gridBagConstraints.gridy = 0;
        gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
        painelPeças.add(marcaPeça, gridBagConstraints);
```

```
todosObjetosCheckBox.setText("Todos os objetos do sinistro");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 1;
gridBagConstraints.gridwidth = 2;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
painelPeças.add(todosObjetosCheckBox, gridBagConstraints);

painelMecânica.setLayout(new java.awt.GridBagLayout());

prazoLabel.setText("Prazo mínimo de garantia");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
painelMecânica.add(prazoLabel, gridBagConstraints);

prazoMínimoGarantia.setColumns(12);
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
painelMecânica.add(prazoMínimoGarantia, gridBagConstraints);

subclassesTabbedPane.addTab("Peça mecânica", painelMecânica);

painelAcessórios.setLayout(new java.awt.GridBagLayout());

corLabel.setText("Cor");
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
painelAcessórios.add(corLabel, gridBagConstraints);

corAcessório.setColumns(12);
```

```
gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 1;
gridBagConstraints.gridy = 0;
gridBagConstraints.anchor = java.awt.GridBagConstraints.WEST;
painelAcessórios.add(corAcessório, gridBagConstraints);

subclassesTabbedPane.addTab("Peça de acessórios", painelAcessórios);

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 2;
gridBagConstraints.gridwidth = 2;
gridBagConstraints.fill = java.awt.GridBagConstraints.BOTH;
gridBagConstraints.weightx = 1.0;
gridBagConstraints.weighty = 1.0;
painelPeças.add(subclassesTabbedPane, gridBagConstraints);

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 3;
gridBagConstraints.fill = java.awt.GridBagConstraints.BOTH;
gridBagConstraints.weightx = 1.0;
gridBagConstraints.weighty = 1.0;
filtros.add(painelPeças, gridBagConstraints);

gridBagConstraints = new java.awt.GridBagConstraints();
gridBagConstraints.gridx = 0;
gridBagConstraints.gridy = 0;
gridBagConstraints.fill = java.awt.GridBagConstraints.HORIZONTAL;
gridBagConstraints.weightx = 1.0;
gridBagConstraints.insets = new java.awt.Insets(4, 6, 4, 6);
getContentPane().add(filtros, gridBagConstraints);

resultados.setEditable(false);
resultados.setLineWrap(true);
resultados.setWrapStyleWord(true);
resultadosScrollPane.setViewportView(resultados);
```

```
        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 1;
        gridBagConstraints.fill = java.awt.GridBagConstraints.BOTH;
        gridBagConstraints.weightx = 1.0;
        gridBagConstraints.weighty = 1.0;
        gridBagConstraints.insets = new java.awt.Insets(4, 6, 4, 6);
        getContentPane().add(resultadosScrollPane, gridBagConstraints);

        pesquisarButton.setText("Pesquisar");
        botoes.add(pesquisarButton);

        limparButton.setText("Limpar Filtros");
        botoes.add(limparButton);

        limparPesquisaButton.setText("Limpar Pesquisa");
        botoes.add(limparPesquisaButton);

        gridBagConstraints = new java.awt.GridBagConstraints();
        gridBagConstraints.gridx = 0;
        gridBagConstraints.gridy = 2;
        gridBagConstraints.fill = java.awt.GridBagConstraints.HORIZONTAL;
        gridBagConstraints.weightx = 1.0;
        gridBagConstraints.insets = new java.awt.Insets(4, 6, 4, 6);
        getContentPane().add(botoes, gridBagConstraints);

        pack();
    }// </editor-fold>//GEN-END:initComponents

    private Double númeroOpcional(JTextField campo, String descrição) {
        String texto = campo.getText().trim();
        if (texto.isEmpty()) return null;
        try {
            double valor = Double.parseDouble(texto.replace(',', '.'));
            if (valor < 0) throw new NumberFormatException();
            return valor;
```

```
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Informe um valor válido e não negativo para " +
descrição + ".");
        }
    }

    private Integer inteiroOpcional(JTextField campo, String descrição) {
        String texto = campo.getText().trim();
        if (texto.isEmpty()) return null;
        try {
            int valor = Integer.parseInt(texto);
            if (valor < 0) throw new NumberFormatException();
            return valor;
        } catch (NumberFormatException e) {
            throw new IllegalArgumentException("Informe um número inteiro não negativo para " +
descrição + ".");
        }
    }

    private void pesquisar() {
        try {
            String textoData = dataMinimaOrçamento.getText().trim();
            Timestamp data = textoData.isEmpty() ? null : obterTimestamp(textoData);
            Boolean atendimentoSelecionado = atendimento24hSim.isSelected() ? Boolean.TRUE
                    : atendimento24hNao.isSelected() ? Boolean.FALSE : null;
            Sinistro.GrauMonta grau = (Sinistro.GrauMonta) grauMonta.getSelectedItem();
            // Somente o filtro da aba selecionada participa da pesquisa. Isso
            // evita que um valor antigo da outra subclasse seja aplicado sem
            // que o usuário perceba.
            Integer prazo = subclassesTabbedPane.getSelectedIndex() == 0
                    ? inteiroOpcional(prazoMínimoGarantia, "prazo mínimo de garantia") : null;
            String cor = subclassesTabbedPane.getSelectedIndex() == 1
                    ? corAcessório.getText().trim() : null;
            if (cor != null && cor.isEmpty()) cor = null;

            Peça.MarcaPeça marca = (Peça.MarcaPeça) marcaPeça.getSelectedItem();
            ArrayList<Orçamento> encontrados = Orçamento.pesquisarOrçamentos(
```

```
                data, marca,
                atendimentoSelecionado, grau, prazo, cor,
                todosObjetosCheckBox.isSelected());
            resultados.setText("");
            if (encontrados.isEmpty()) {
                resultados.setText("Nenhum orçamento encontrado para os filtros informados.");
                return;
            }
            for (Orçamento orçamento : encontrados) {
                if (resultados.getDocument().getLength() > 0) resultados.append("\n\n");
                resultados.append(orçamento.toStringFull());
            }
        } catch (IllegalArgumentException e) {
            JOptionPane.showMessageDialog(this,
                    e.getMessage() == null ? "Informe a data no formato AAAA-MM-DD HH:MM:SS." :
e.getMessage(),
                    "Filtro inválido", JOptionPane.ERROR_MESSAGE);
        }
    }

    private Timestamp obterTimestamp(String texto) {
        try {
            return Timestamp.valueOf(texto.length() == 10 ? texto + " 00:00:00" : texto);
        } catch (IllegalArgumentException e) {
            throw new IllegalArgumentException("Informe a data no formato AAAA-MM-DD
HH:MM:SS.");
        }
    }

    private void limparFiltros() {
        dataMinimaOrçamento.setText("");
        marcaPeça.setSelectedIndex(-1);
        atendimento24hGrupo.clearSelection();
        grauMonta.setSelectedIndex(-1);
        prazoMínimoGarantia.setText("");
        corAcessório.setText("");
        todosObjetosCheckBox.setSelected(false);
```

```
        subclassesTabbedPane.setSelectedIndex(0);
    }

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JPanel atendimento24h;
    private javax.swing.ButtonGroup atendimento24hGrupo;
    private javax.swing.JRadioButton atendimento24hNao;
    private javax.swing.JRadioButton atendimento24hSim;
    private javax.swing.JLabel atendimentoLabel;
    private javax.swing.JPanel botoes;
    private javax.swing.JTextField corAcessório;
    private javax.swing.JLabel corLabel;
    private javax.swing.JLabel dataLabel;
    private javax.swing.JTextField dataMinimaOrçamento;
    private javax.swing.JPanel filtros;
    private javax.swing.JLabel grauLabel;
    private javax.swing.JComboBox grauMonta;
    private javax.swing.JButton limparButton;
    private javax.swing.JButton limparPesquisaButton;
    private javax.swing.JLabel marcaLabel;
    private javax.swing.JComboBox marcaPeça;
    private javax.swing.JPanel painelAcessórios;
    private javax.swing.JPanel painelMecânica;
    private javax.swing.JPanel painelOrçamento;
    private javax.swing.JPanel painelPeças;
    private javax.swing.JPanel painelSeguradora;
    private javax.swing.JPanel painelSinistro;
    private javax.swing.JButton pesquisarButton;
    private javax.swing.JLabel prazoLabel;
    private javax.swing.JTextField prazoMínimoGarantia;
    private javax.swing.JTextArea resultados;
    private javax.swing.JScrollPane resultadosScrollPane;
    private javax.swing.JTabbedPane subclassesTabbedPane;
    private javax.swing.JCheckBox todosObjetosCheckBox;
    // End of variables declaration//GEN-END:variables

}
```

$$$ src.interfaces.JanelaSistema

```
package interfaces;

import controles.ControladorCadastroSeguradoras;
import controles.ControladorCadastroPeças;
import controles.ControladorCadastroSinistros;
import controles.ControladorCadastroOrçamentos;
import persistência.BD;

public class JanelaSistema extends javax.swing.JFrame {

    public JanelaSistema() {
        BD.criaConexao();
        initComponents();
        setLocationByPlatform(true);
    }

    @SuppressWarnings("unchecked")

    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {

        seguradoras_orçamentoMenuBar = new javax.swing.JMenuBar();
        peçaMenu = new javax.swing.JMenu();
        cadastrar_peçaMenuItem = new javax.swing.JMenuItem();
        seguradoraMenu = new javax.swing.JMenu();
        cadastrarSeguradoraItemMenu = new javax.swing.JMenuItem();
        sinistroMenu = new javax.swing.JMenu();
        cadastrarSinistroMenuItem = new javax.swing.JMenuItem();
        orçamentoMenu = new javax.swing.JMenu();
        cadastrar_orçamentoMenuItem = new javax.swing.JMenuItem();
        pesquisar_orçamentoMenuItem = new javax.swing.JMenuItem();

        setDefaultCloseOperation(javax.swing.WindowConstants.DISPOSE_ON_CLOSE);
        setTitle("Orçamentos da Seguradora");
        addWindowListener(new java.awt.event.WindowAdapter() {
```

```
    public void windowClosed(java.awt.event.WindowEvent evt) {
        terminarSistema(evt);
    }
});

peçaMenu.setText("Peça");

cadastrar_peçaMenuItem.setText("Cadastrar");
cadastrar_peçaMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarPeça(evt);
    }
});
peçaMenu.add(cadastrar_peçaMenuItem);

seguradoras_orçamentoMenuBar.add(peçaMenu);

seguradoraMenu.setText("Seguradora");

cadastrarSeguradoraItemMenu.setText("Cadastrar");
cadastrarSeguradoraItemMenu.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSeguradora(evt);
    }
});
seguradoraMenu.add(cadastrarSeguradoraItemMenu);

seguradoras_orçamentoMenuBar.add(seguradoraMenu);

sinistroMenu.setText("Sinistro");

cadastrarSinistroMenuItem.setText("Cadastrar");
cadastrarSinistroMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarSinistro(evt);
    }
});
```

```
sinistroMenu.add(cadastrarSinistroMenuItem);

seguradoras_orçamentoMenuBar.add(sinistroMenu);

orçamentoMenu.setText("Orçamento");

cadastrar_orçamentoMenuItem.setText("Cadastrar");
cadastrar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        cadastrarOrçamentos(evt);
    }
});
orçamentoMenu.add(cadastrar_orçamentoMenuItem);

pesquisar_orçamentoMenuItem.setText("Pesquisar");
pesquisar_orçamentoMenuItem.addActionListener(new java.awt.event.ActionListener() {
    public void actionPerformed(java.awt.event.ActionEvent evt) {
        pesquisarOrçamentos(evt);
    }
});
orçamentoMenu.add(pesquisar_orçamentoMenuItem);

seguradoras_orçamentoMenuBar.add(orçamentoMenu);

setJMenuBar(seguradoras_orçamentoMenuBar);

javax.swing.GroupLayout layout = new javax.swing.GroupLayout(getContentPane());
getContentPane().setLayout(layout);
layout.setHorizontalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGap(0, 400, Short.MAX_VALUE)
);
layout.setVerticalGroup(
    layout.createParallelGroup(javax.swing.GroupLayout.Alignment.LEADING)
    .addGap(0, 279, Short.MAX_VALUE)
);
```

```
        pack();
    }// </editor-fold>//GEN-END:initComponents

    private void cadastrarSeguradora(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_cadastrarSeguradora
        new ControladorCadastroSeguradoras(this);
    }//GEN-LAST:event_cadastrarSeguradora

    private void terminarSistema(java.awt.event.WindowEvent evt) {//GEN-
FIRST:event_terminarSistema
        BD.fechaConexao();
        System.exit(0);
    }//GEN-LAST:event_terminarSistema

    private void pesquisarOrçamentos(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_pesquisarOrçamentos
        JanelaPesquisaOrçamentos janela = new JanelaPesquisaOrçamentos();
        janela.setVisible(true);
        janela.toFront();
        janela.requestFocus();
    }//GEN-LAST:event_pesquisarOrçamentos

    private void cadastrarPeça(java.awt.event.ActionEvent evt) {//GEN-FIRST:event_cadastrarPeça
        new ControladorCadastroPeças(this);
    }//GEN-LAST:event_cadastrarPeça

    private void cadastrarOrçamentos(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_cadastrarOrçamentos
        new ControladorCadastroOrçamentos();
    }//GEN-LAST:event_cadastrarOrçamentos

    private void cadastrarSinistro(java.awt.event.ActionEvent evt) {//GEN-
FIRST:event_cadastrarSinistro
        new ControladorCadastroSinistros(this);
    }//GEN-LAST:event_cadastrarSinistro

    public static void main(String args[]) {
        java.awt.EventQueue.invokeLater(new Runnable() {
```

```
            public void run() {
                new JanelaSistema().setVisible(true);
            }
        });
    }

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private javax.swing.JMenuItem cadastrar_orçamentoMenuItem;
    private javax.swing.JMenuItem cadastrar_peçaMenuItem;
    private javax.swing.JMenuItem cadastrarSeguradoraItemMenu;
    private javax.swing.JMenuItem cadastrarSinistroMenuItem;
    private javax.swing.JMenu orçamentoMenu;
    private javax.swing.JMenuItem pesquisar_orçamentoMenuItem;
    private javax.swing.JMenu peçaMenu;
    private javax.swing.JMenu seguradoraMenu;
    private javax.swing.JMenuBar seguradoras_orçamentoMenuBar;
    private javax.swing.JMenu sinistroMenu;
    // End of variables declaration//GEN-END:variables

}
```

$$$ src.interfaces.PainelPeçaAcessórios

```
package interfaces;

import entidades.PeçaAcessório;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import javax.swing.JCheckBox;
import javax.swing.JComboBox;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class PainelPeçaAcessórios extends JPanel {
    public PainelPeçaAcessórios() {
        initComponents();
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {
        categoriaLabel = new JLabel("Categoria do acessório");
        corLabel = new JLabel("Cor");
        funileiroProprioLabel = new JLabel("Funileiro próprio");
        categoriaComboBox = new JComboBox<>(PeçaAcessório.CategoriaAcessório.values());
        corTextField = new JTextField(10);
        funileiroProprioCheckBox = new JCheckBox("Sim");

        setLayout(new GridBagLayout());

        GridBagConstraints categoriaLabelConstraints = new GridBagConstraints();
        categoriaLabelConstraints.gridx = 0;
        categoriaLabelConstraints.gridy = 0;
        categoriaLabelConstraints.insets = new Insets(1, 5, 10, 5);
        categoriaLabelConstraints.anchor = GridBagConstraints.WEST;
        add(categoriaLabel, categoriaLabelConstraints);
```

```
    GridBagConstraints categoriaComboBoxConstraints = new GridBagConstraints();
    categoriaComboBoxConstraints.gridx = 1;
    categoriaComboBoxConstraints.gridy = 0;
    categoriaComboBoxConstraints.insets = new Insets(1, 5, 10, 5);
    categoriaComboBoxConstraints.anchor = GridBagConstraints.WEST;
    add(categoriaComboBox, categoriaComboBoxConstraints);

    GridBagConstraints corLabelConstraints = new GridBagConstraints();
    corLabelConstraints.gridx = 0;
    corLabelConstraints.gridy = 1;
    corLabelConstraints.insets = new Insets(1, 5, 10, 5);
    corLabelConstraints.anchor = GridBagConstraints.WEST;
    add(corLabel, corLabelConstraints);

    GridBagConstraints corTextFieldConstraints = new GridBagConstraints();
    corTextFieldConstraints.gridx = 1;
    corTextFieldConstraints.gridy = 1;
    corTextFieldConstraints.insets = new Insets(1, 5, 10, 5);
    corTextFieldConstraints.anchor = GridBagConstraints.WEST;
    add(corTextField, corTextFieldConstraints);

    GridBagConstraints funileiroProprioLabelConstraints = new GridBagConstraints();
    funileiroProprioLabelConstraints.gridx = 0;
    funileiroProprioLabelConstraints.gridy = 2;
    funileiroProprioLabelConstraints.insets = new Insets(1, 5, 10, 5);
    funileiroProprioLabelConstraints.anchor = GridBagConstraints.WEST;
    add(funileiroProprioLabel, funileiroProprioLabelConstraints);

    GridBagConstraints funileiroProprioCheckBoxConstraints = new GridBagConstraints();
    funileiroProprioCheckBoxConstraints.gridx = 1;
    funileiroProprioCheckBoxConstraints.gridy = 2;
    funileiroProprioCheckBoxConstraints.insets = new Insets(1, 5, 10, 5);
    funileiroProprioCheckBoxConstraints.anchor = GridBagConstraints.WEST;
    add(funileiroProprioCheckBox, funileiroProprioCheckBoxConstraints);
}// </editor-fold>//GEN-END:initComponents
```

```
    public PeçaAcessório.CategoriaAcessório getCategoriaAcessório() {
        return (PeçaAcessório.CategoriaAcessório) categoriaComboBox.getSelectedItem();
    }
    public String getCor() { return corTextField.getText().trim(); }
    public boolean isFunileiroProprio() { return funileiroProprioCheckBox.isSelected(); }
    public void setCategoriaAcessório(PeçaAcessório.CategoriaAcessório valor)
{ categoriaComboBox.setSelectedItem(valor); }
    public void setCor(String valor) { corTextField.setText(valor == null ? "" : valor); }
    public void setFunileiroProprio(boolean valor) { funileiroProprioCheckBox.setSelected(valor); }
    public void limparCampos() { categoriaComboBox.setSelectedIndex(-1); setCor(null);
setFunileiroProprio(false); }

    // Variables declaration - do not modify//GEN-BEGIN:variables
    private JComboBox<PeçaAcessório.CategoriaAcessório> categoriaComboBox;
    private JTextField corTextField;
    private JCheckBox funileiroProprioCheckBox;
    private JLabel categoriaLabel, corLabel, funileiroProprioLabel;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.interfaces.PainelPeçaMecânica

```
package interfaces;

import entidades.PeçaMecânica;
import java.awt.GridBagConstraints;
import java.awt.GridBagLayout;
import java.awt.Insets;
import javax.swing.JComboBox;
import javax.swing.JCheckBox;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;

public class PainelPeçaMecânica extends JPanel {
    public PainelPeçaMecânica() {
        initComponents();
    }

    @SuppressWarnings("unchecked")
    // <editor-fold defaultstate="collapsed" desc="Generated Code">//GEN-BEGIN:initComponents
    private void initComponents() {
        categoriaLabel = new JLabel("Categoria mecânica");
        prazoGarantiaLabel = new JLabel("Prazo de garantia (dias)");
        mecanicoProprioLabel = new JLabel("Mecânico próprio");
        categoriaComboBox = new JComboBox<>(PeçaMecânica.CategoriaMecânica.values());
        prazoGarantiaTextField = new JTextField(10);
        mecanicoProprioCheckBox = new JCheckBox("Sim");

        setLayout(new GridBagLayout());

        GridBagConstraints categoriaLabelConstraints = new GridBagConstraints();
        categoriaLabelConstraints.gridx = 0;
        categoriaLabelConstraints.gridy = 0;
        categoriaLabelConstraints.insets = new Insets(1, 5, 10, 5);
        categoriaLabelConstraints.anchor = GridBagConstraints.WEST;
        add(categoriaLabel, categoriaLabelConstraints);
```

```
    GridBagConstraints categoriaComboBoxConstraints = new GridBagConstraints();
    categoriaComboBoxConstraints.gridx = 1;
    categoriaComboBoxConstraints.gridy = 0;
    categoriaComboBoxConstraints.insets = new Insets(1, 5, 10, 5);
    categoriaComboBoxConstraints.anchor = GridBagConstraints.WEST;
    add(categoriaComboBox, categoriaComboBoxConstraints);

    GridBagConstraints prazoGarantiaLabelConstraints = new GridBagConstraints();
    prazoGarantiaLabelConstraints.gridx = 0;
    prazoGarantiaLabelConstraints.gridy = 1;
    prazoGarantiaLabelConstraints.insets = new Insets(1, 5, 10, 5);
    prazoGarantiaLabelConstraints.anchor = GridBagConstraints.WEST;
    add(prazoGarantiaLabel, prazoGarantiaLabelConstraints);

    GridBagConstraints prazoGarantiaTextFieldConstraints = new GridBagConstraints();
    prazoGarantiaTextFieldConstraints.gridx = 1;
    prazoGarantiaTextFieldConstraints.gridy = 1;
    prazoGarantiaTextFieldConstraints.insets = new Insets(1, 5, 10, 5);
    prazoGarantiaTextFieldConstraints.anchor = GridBagConstraints.WEST;
    add(prazoGarantiaTextField, prazoGarantiaTextFieldConstraints);

    GridBagConstraints mecanicoProprioLabelConstraints = new GridBagConstraints();
    mecanicoProprioLabelConstraints.gridx = 0;
    mecanicoProprioLabelConstraints.gridy = 2;
    mecanicoProprioLabelConstraints.insets = new Insets(1, 5, 10, 5);
    mecanicoProprioLabelConstraints.anchor = GridBagConstraints.WEST;
    add(mecanicoProprioLabel, mecanicoProprioLabelConstraints);

    GridBagConstraints mecanicoProprioCheckBoxConstraints = new GridBagConstraints();
    mecanicoProprioCheckBoxConstraints.gridx = 1;
    mecanicoProprioCheckBoxConstraints.gridy = 2;
    mecanicoProprioCheckBoxConstraints.insets = new Insets(1, 5, 10, 5);
    mecanicoProprioCheckBoxConstraints.anchor = GridBagConstraints.WEST;
    add(mecanicoProprioCheckBox, mecanicoProprioCheckBoxConstraints);
}// </editor-fold>//GEN-END:initComponents
```

```
    public PeçaMecânica.CategoriaMecânica getCategoriaMecânica() {
        return (PeçaMecânica.CategoriaMecânica) categoriaComboBox.getSelectedItem();
    }
    public Integer getPrazoGarantia() {
        String valor = prazoGarantiaTextField.getText().trim();
        return valor.isEmpty() ? null : Integer.valueOf(valor);
    }
    public boolean isMecanicoProprio() { return mecanicoProprioCheckBox.isSelected(); }
    public void setCategoriaMecânica(PeçaMecânica.CategoriaMecânica valor)
{ categoriaComboBox.setSelectedItem(valor); }
    public void setPrazoGarantia(Integer valor) { prazoGarantiaTextField.setText(valor == null ? "" :
valor.toString()); }
    public void setMecanicoProprio(boolean valor)
{ mecanicoProprioCheckBox.setSelected(valor); }
    public void limparCampos() { categoriaComboBox.setSelectedIndex(-1); setPrazoGarantia(null);
setMecanicoProprio(false); }


    // Variables declaration - do not modify//GEN-BEGIN:variables
    private JComboBox<PeçaMecânica.CategoriaMecânica> categoriaComboBox;
    private JTextField prazoGarantiaTextField;
    private JCheckBox mecanicoProprioCheckBox;
    private JLabel categoriaLabel, prazoGarantiaLabel, mecanicoProprioLabel;
    // End of variables declaration//GEN-END:variables
}
```

$$$ src.persistência.BD

```
package persistência;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class BD {

   static final String URL_BD = "jdbc:mysql://localhost/banco?
useUnicode=true&characterEncoding=utf8&connectionCollation=utf8mb4_unicode_ci&serverTim
ezone=UTC&useSSL=false&allowPublicKeyRetrieval=true";
   static final String USUARIO = "root";
   static final String SENHA = "admin";
   public static Connection conexao = null;

   public static void criaConexao() {
      try {
         conexao = DriverManager.getConnection(URL_BD, USUARIO, SENHA);
         atualizarEsquemaEtapa3();
         atualizarEsquemaEtapa4();
      } catch (SQLException excecaoSql) {
         excecaoSql.printStackTrace();
      }
   }

   private static void atualizarEsquemaEtapa3() throws SQLException {
      boolean temPrecoAntigo;
      boolean temPrecoNovo;
      String sqlColuna = "SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS "
            + "WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'peças' AND
COLUMN_NAME = ?";
      try (PreparedStatement comando = conexao.prepareStatement(sqlColuna)) {
         comando.setString(1, "preco");
```

```
            try (ResultSet resultado = comando.executeQuery()) {
                resultado.next();
                temPrecoAntigo = resultado.getInt(1) > 0;
            }
            comando.setString(1, "preço");
            try (ResultSet resultado = comando.executeQuery()) {
                resultado.next();
                temPrecoNovo = resultado.getInt(1) > 0;
            }
        }

        if (temPrecoAntigo && !temPrecoNovo) {
            try (Statement comando = conexao.createStatement()) {
                comando.executeUpdate("ALTER TABLE peças CHANGE COLUMN preco `preço`
DECIMAL(10,2)");
            }
        }

        try (Statement comando = conexao.createStatement()) {
            garantirColuna(comando, "peças", "sob_encomenda", "BOOLEAN NOT NULL DEFAULT
FALSE");
            removerColunaSeExistir(comando, "peças", "mao_obra_propria");
            comando.executeUpdate("CREATE TABLE IF NOT EXISTS peça_mecânica ("
                    + "categoria_mecânica VARCHAR(40) NOT NULL, prazo_garantia INT,
mecanico_proprio BOOLEAN NOT NULL DEFAULT FALSE, "
                    + "codigoId INT NOT NULL PRIMARY KEY, FOREIGN KEY (codigoId)
REFERENCES peças(codigo) ON DELETE CASCADE)");
            comando.executeUpdate("CREATE TABLE IF NOT EXISTS peça_acessórios ("
                    + "categoria_acessorio VARCHAR(40) NOT NULL, cor VARCHAR(20),
funileiro_proprio BOOLEAN NOT NULL DEFAULT FALSE, "
                    + "codigoId INT NOT NULL PRIMARY KEY, FOREIGN KEY (codigoId)
REFERENCES peças(codigo) ON DELETE CASCADE)");
            renomearCategoriaAcessorioLegada(comando);
            garantirColuna(comando, "peça_mecânica", "mecanico_proprio", "BOOLEAN NOT NULL
DEFAULT FALSE");
            garantirColuna(comando, "peça_acessórios", "funileiro_proprio", "BOOLEAN NOT NULL
DEFAULT FALSE");
            garantirCodigoUnico(comando, "peça_mecânica");
            garantirCodigoUnico(comando, "peça_acessórios");
```

```
        }
    }

    /** Cria a tabela de orçamentos quando o banco existente ainda está na etapa 3. */
    private static void atualizarEsquemaEtapa4() throws SQLException {
        try (Statement comando = conexao.createStatement()) {
            comando.executeUpdate("CREATE TABLE IF NOT EXISTS `Orçamentos` ("
                    + "data TIMESTAMP NOT NULL, sinistro INT NOT NULL, seguradora
VARCHAR(50) NOT NULL, "
                    + "status VARCHAR(10) NOT NULL, valor_total DECIMAL(10,2) NOT NULL, "
                    + "possui_danos_ocultos BOOLEAN NOT NULL DEFAULT FALSE, "
                    + "PRIMARY KEY (sinistro, seguradora), "
                    + "FOREIGN KEY (sinistro) REFERENCES sinistros(id), "
                    + "FOREIGN KEY (seguradora) REFERENCES seguradoras(nome))");
            // Converte a coluna criada na etapa anterior para preservar também hora, minuto e segundo.
            try (PreparedStatement consulta = conexao.prepareStatement(
                    "SELECT DATA_TYPE FROM INFORMATION_SCHEMA.COLUMNS "
                    + "WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME =
'Orçamentos' AND COLUMN_NAME = 'data'")) {
                try (ResultSet resultado = consulta.executeQuery()) {
                    if (resultado.next() && "date".equalsIgnoreCase(resultado.getString(1))) {
                        comando.executeUpdate("ALTER TABLE `Orçamentos` MODIFY COLUMN data
TIMESTAMP NOT NULL");
                    }
                }
            }
        }
    }

    private static void garantirCodigoUnico(Statement comando, String tabela) throws
SQLException {
        String sql = "SELECT COUNT(*) FROM INFORMATION_SCHEMA.STATISTICS
WHERE TABLE_SCHEMA=DATABASE() "
                + "AND TABLE_NAME='" + tabela + "' AND COLUMN_NAME='codigoId' AND
NON_UNIQUE=0";
        try (ResultSet resultado = comando.executeQuery(sql)) {
            resultado.next();
            if (resultado.getInt(1) == 0) {
```

```
            // Dados legados podem conter várias linhas para a mesma peça; preserva uma antes da chave única.
            String campos = tabela.equals("peça_mecânica")
                    ?
"MAX(categoria_mecânica),MAX(prazo_garantia),MAX(mecanico_proprio),codigoId"
                    : "MAX(categoria_acessorio),MAX(cor),MAX(funileiro_proprio),codigoId";
            String colunas = tabela.equals("peça_mecânica")
                    ? "categoria_mecânica,prazo_garantia,mecanico_proprio,codigoId"
                    : "categoria_acessorio,cor,funileiro_proprio,codigoId";
            comando.executeUpdate("CREATE TEMPORARY TABLE tmp_subclasse AS SELECT " + campos
                    + " FROM `" + tabela + "` GROUP BY codigoId");
            comando.executeUpdate("DELETE FROM `" + tabela + "`");
            comando.executeUpdate("INSERT INTO `" + tabela + "` (" + colunas + ") SELECT * FROM tmp_subclasse");
            comando.executeUpdate("DROP TEMPORARY TABLE tmp_subclasse");
            comando.executeUpdate("ALTER TABLE `" + tabela + "` ADD UNIQUE KEY uq_" + tabela + "_codigoId (codigoId)");
        }
    }
}

private static void garantirColuna(Statement comando, String tabela, String coluna, String definicao) throws SQLException {
    String sql = "SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA=DATABASE() AND TABLE_NAME='" + tabela + "' AND COLUMN_NAME='" + coluna + "'";
    try (ResultSet resultado = comando.executeQuery(sql)) {
        resultado.next();
        if (resultado.getInt(1) == 0) comando.executeUpdate("ALTER TABLE `" + tabela + "` ADD COLUMN `" + coluna + "` " + definicao);
    }
}

private static void removerColunaSeExistir(Statement comando, String tabela, String coluna) throws SQLException {
    String sql = "SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA=DATABASE() AND TABLE_NAME='" + tabela + "' AND COLUMN_NAME='" + coluna + "'";
    try (ResultSet resultado = comando.executeQuery(sql)) {
```

```
            resultado.next();
            if (resultado.getInt(1) > 0) comando.executeUpdate("ALTER TABLE `" + tabela + "` DROP COLUMN `" + coluna + "`");
        }
    }

    private static void renomearCategoriaAcessorioLegada(Statement comando) throws SQLException {
        String sql = "SELECT COUNT(*) FROM INFORMATION_SCHEMA.COLUMNS WHERE TABLE_SCHEMA=DATABASE() AND TABLE_NAME='peça_acessórios' AND COLUMN_NAME='categoria_lataria'";
        try (ResultSet resultado = comando.executeQuery(sql)) {
            resultado.next();
            if (resultado.getInt(1) > 0) {
                comando.executeUpdate("ALTER TABLE peça_acessórios CHANGE COLUMN categoria_lataria categoria_acessorio VARCHAR(40) NOT NULL");
            }
        }
    }

    public static void fechaConexao() {
        try {
            if (conexao != null) {
                conexao.close();
            }
        } catch (SQLException excecaoSql) {
            excecaoSql.printStackTrace();
        }
    }
}
```